

MongoDB

120 Interview Questions & Answers

What is MongoDB & why use it

SQL vs NoSQL (when to use MongoDB)

Documents, Collections & BSON

_id field & ObjectId

CRUD operations (insert, find, update, delete)

Query operators (\$eq, \$gt, \$lt, \$in)

Logical operators (\$and, \$or, \$not)

Projection (select specific fields)

Sorting & limiting results

Pagination (skip vs range query)

Indexing (why & how)

Types of indexes (single, compound, text)

Unique index & sparse index

TTL index (auto delete data)

Aggregation pipeline (basics)

\$match, \$group, \$sort, \$limit

\$lookup (joins in MongoDB)

\$unwind & array handling

Schema design (embedding vs referencing)

One-to-many & many-to-many relationships

Denormalization (pros & cons)

Transactions in MongoDB

ACID properties in MongoDB

→ All 64 questions answered with production code ←

→ Follow for Next:Final full-stack 950+ (Q&A) ←

■ Save • ■ Comment (JS950) to get the full PDF — dropping tomorrow ■ • ■■ Repost

Kaushal Singh

Full-Stack Dev | MERN + React Native | Next.js

1M+ Impressions | Follow for Final full-stack 950+ (Q&A) Interview Prep

■ Section 7/10 | 80 Q&A | Next: MongoDB 120 Q | Comment 'JS950'

What is MongoDB? SQL vs NoSQL?

■ Answer

MongoDB is a document-oriented NoSQL database that stores data as JSON-like documents.

SQL (Relational) Databases:

- Structured: tables, rows, columns with fixed schema
- ACID transactions (strong consistency)
- JOINS for related data
- Vertical scaling (bigger machine)
- Examples: PostgreSQL, MySQL, SQLite

NoSQL (MongoDB) Databases:

- Flexible: documents can have different fields
- Horizontal scaling (more machines — sharding)
- Embedded documents instead of JOINS
- Schema-less (but you should still use Mongoose schemas!)

When to use MongoDB:

- Rapidly changing data structures
- Hierarchical data (product with many attributes)
- High write throughput, horizontal scaling
- Content management, catalogs, user profiles

When to use SQL:

- Complex relationships with many JOINS
- Financial data requiring ACID transactions
- Well-defined, stable schema

```
1 // SQL: Users and Posts (separate tables + JOIN)
2 // SELECT u.name, p.title FROM users u
3 // JOIN posts p ON p.user_id = u.id
4 // WHERE u.id = 1
5
6 // MongoDB: User with embedded/referenced posts
7 // Embedded (denormalized):
8 const user = {
9   _id: ObjectId('...'),
10  name: 'Kaushal',
11  email: 'kaushal@example.com',
12  // Small, bounded array — embed!
13  skills: ['React', 'Node.js', 'MongoDB'],
14  address: { city: 'Mumbai', state: 'Maharashtra' } // always with user
15 }
16
17 // Referenced (normalized):
18 const post = {
19   _id: ObjectId('...'),
```

```
20   title: 'Why I love MongoDB',
```

```
21   userId: ObjectId('...'), // reference to user
22   content: '...',
23   tags: ['mongodb', 'nosql']
24 }
25
26 // MongoDB query (no JOINS needed if embedded):
27 db.users.find({ 'address.city': 'Mumbai' })
```

■ Interview Tip:

Choose the right database for your use case. MongoDB excels at flexibility and scale. PostgreSQL excels at complex queries and transactions. Many production systems use both (polyglot persistence).

Q2 What is Mongoose? Why use it with MongoDB?

■ Answer

Mongoose is an ODM (Object Document Mapper) for MongoDB and Node.js. It provides schema validation, middleware, and a clean API on top of the MongoDB driver.

What Mongoose adds:

- Schema definition — define structure and types
- Validation — built-in and custom validators
- Middleware (hooks) — pre/post hooks on operations
- Population — populate referenced documents
- Virtuals — computed properties
- Query builder — chainable API
- TypeScript support

Without Mongoose:

- No validation — any data goes in
- Manual type conversion
- Verbose query code
- No hooks for password hashing, etc.

```

1  const mongoose = require('mongoose')
2  const bcrypt = require('bcrypt')
3
4  // Define schema
5  const userSchema = new mongoose.Schema({
6    name: { type: String, required: [true, 'Name is required'], trim: true, minlength: 2 },
7    email: { type: String, required: true, unique: true, lowercase: true,
8      match: [/^\S+@\S+\.\S+$/, 'Invalid email'] },
9    password: { type: String, required: true, minlength: 8, select: false },
10   role: { type: String, enum: ['user', 'admin', 'editor'], default: 'user' },
11   age: { type: Number, min: 0, max: 150 },
12   active: { type: Boolean, default: true },
13   createdAt: { type: Date, default: Date.now }
14 }, {
15   timestamps: true, // auto-add createdAt & updatedAt
16   toJSON: { virtuals: true }
17 })
18
19 // Virtual — computed property (not stored in DB)
20 userSchema.virtual('fullName').get(function() {
21   return `${this.firstName} ${this.lastName}`
22 })
23
24 // Pre-save hook — hash password before saving
25 userSchema.pre('save', async function(next) {
26   if (!this.isModified('password')) return next()
27   this.password = await bcrypt.hash(this.password, 12)
28   next()
29 })

```



```
30
31 // Instance method – verify password
32 userSchema.methods.comparePassword = async function(candidatePassword) {
33   return bcrypt.compare(candidatePassword, this.password)
34 }
35
36 // Static method – find active users
37 userSchema.statics.findActive = function() {
38   return this.find({ active: true })
39 }
40
41 const User = mongoose.model('User', userSchema)
42 module.exports = User
```

■ Interview Tip:

Mongoose pre-save hooks are the cleanest way to hash passwords — the hashing happens automatically whenever a user is created or password changed. Never hash in the controller — you'll inevitably forget some code path.

How does MongoDB indexing work?

■ Answer

Indexes are data structures that improve query performance by avoiding full collection scans. Without indexes, MongoDB reads EVERY document to find matches.

Index types:

- Single field — { email: 1 } ascending, { -1 } descending
- Compound — { firstName: 1, lastName: 1 } — order matters!
- Multikey — auto for array fields
- Text — for full-text search (\$text queries)
- Geospatial — for location queries (\$near, \$geoWithin)
- Sparse — only indexes documents that have the field
- TTL — auto-delete documents after N seconds
- Unique — enforces uniqueness constraint

Performance impact:

- Query goes from $O(n)$ to $O(\log n)$ with index
- Trade-off: faster reads, slower writes (index must update)

```

1  const mongoose = require('mongoose')
2
3  const productSchema = new mongoose.Schema({
4    name: String,
5    price: Number,
6    category: String,
7    tags: [String],           // multikey index auto for arrays
8    description: String,      // for text search
9    location: {               // for geospatial
10     type: { type: String, default: 'Point' },
11     coordinates: [Number]    // [longitude, latitude]
12   },
13   expiresAt: Date,          // for TTL index
14   sku: String
15 })
16
17 // Single field index
18 productSchema.index({ category: 1 })
19
20 // Compound index (for queries filtering both)
21 productSchema.index({ category: 1, price: -1 })
22 // ESR Rule: Equality fields first, Sort fields next, Range fields last
23
24 // Unique index
25 productSchema.index({ sku: 1 }, { unique: true })
26
27 // Text index for search
28 productSchema.index({ name: 'text', description: 'text' })
29 // Query: Product.find({ $text: { $search: 'wireless keyboard' } })

```

```
31 // Geospatial index
32 productSchema.index({ location: '2dsphere' })
33 // Query: stores within 5km radius
34
35 // TTL index – auto-delete expired sessions after 1 hour
36 const sessionSchema = new mongoose.Schema({ expiresAt: Date })
37 sessionSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 })
38
39 // Check index usage – use explain!
40 // const result = await Product.find({ category: 'Electronics' }).explain('executionStats')
41 // Look for: IXSCAN (good) vs COLLSCAN (bad – no index used!)
```

■ Interview Tip:

The ESR (Equality, Sort, Range) rule is key for compound index order. If you query { status: 'active' } and sort by { createdAt: -1 }, create index { status: 1, createdAt: -1 } for maximum efficiency.

Q4 What is aggregation pipeline in MongoDB?

■ Answer

Aggregation pipeline processes documents through stages to compute results. More powerful than basic `find()` — can transform, group, join, and compute.

Key pipeline stages:

- `$match` — filter documents (use EARLY for performance!)
- `$project` — reshape documents (include/exclude/rename fields)
- `$group` — group and aggregate (sum, avg, count)
- `$sort` — sort documents
- `$limit` / `$skip` — pagination
- `$lookup` — JOIN with another collection
- `$unwind` — flatten arrays
- `$addFields` — add computed fields
- `$facet` — multiple aggregations in one pipeline

Performance tip:

- Put `$match` and `$limit` EARLY to reduce documents in pipeline

```
1 // Sales analytics aggregation
2 const salesStats = await Order.aggregate([
3   // Stage 1: Filter (use index!)
4   { $match: {
5     status: 'completed',
6     createdAt: { $gte: new Date('2024-01-01') }
7   }},
8
9   // Stage 2: Group by category with stats
10  { $group: {
11    _id: '$category',
12    totalRevenue: { $sum: { $multiply: ['$price', '$quantity'] } },
13    avgOrderValue: { $avg: '$total' },
14    orderCount: { $sum: 1 },
15    uniqueCustomers: { $addToSet: '$customerId' }
16  }},
17
18  // Stage 3: Add computed fields
19  { $addFields: {
20    uniqueCustomerCount: { $size: '$uniqueCustomers' }
21  }},
22
23  // Stage 4: Project final shape
24  { $project: {
25    category: '$_id', _id: 0,
26    totalRevenue: { $round: ['$totalRevenue', 2] },
27    avgOrderValue: { $round: ['$avgOrderValue', 2] },
28    orderCount: 1, uniqueCustomerCount: 1
29  }},
30 ])
```



```

31 // Stage 5: Sort by revenue
32 { $sort: { totalRevenue: -1 } },
33
34 // Stage 6: Limit
35 { $limit: 10 }
36 ])
37
38 // $lookup – JOIN users with their orders
39 await User.aggregate([
40   { $lookup: {
41     from: 'orders',           // collection to join
42     localField: '_id',        // User._id
43     foreignField: 'userId',    // Order.userId
44     as: 'orders'              // output array field
45   }},
46   { $addFields: { orderCount: { $size: '$orders' } } },
47   { $match: { orderCount: { $gt: 0 } } }
48 ])

```

■ Interview Tip:

Put \$match as early as possible to filter documents before expensive stages. An aggregation that filters at the end processes ALL documents through every stage — very slow on large collections.

Embedding vs Referencing in MongoDB schema design?

■ Answer

Schema design in MongoDB is about choosing the right relationship strategy. Unlike SQL, MongoDB gives you a choice between embedding and referencing.

Embedding (denormalized):

- Store related data in the SAME document
- Fast reads — one query gets everything
- Good for: data always read together, small bounded sets
- Problem: document size limit (16MB), duplication

Referencing (normalized):

- Store a reference (ObjectId) to another document
- Similar to SQL foreign keys
- Good for: large related datasets, many-to-many
- Requires \$lookup (JOIN) or populate() for reads

The Rule of Thumb:

- Embed if: few items, always needed together, don't change often
- Reference if: many items, independently accessed, frequently updated

```

1  // Embedding – good for blog post with comments (few, always shown together)
2  const postSchema = {
3    title: String, content: String,
4    author: {                // Embedded author info (denormalized)
5      name: String,          // duplicated from User – trade-off!
6      avatar: String
7    },
8    tags: [String],          // simple array – embed!
9    comments: [{             // Embedded comments – shown with post
10     text: String, author: String, createdAt: Date
11   }]
12   // Problem: if post has 100k comments, document gets huge!
13 }
14
15 // Hybrid – embed recent comments, reference old ones
16 const betterPostSchema = {
17   title: String, content: String,
18   authorId: ObjectId,        // reference – user accessed separately
19   commentCount: Number,      // denormalized count (avoid $lookup for count)
20   recentComments: [{ text: String, author: String, date: Date }], // last 5
21 }
22
23 // Referencing – users and orders (many orders per user)
24 const orderSchema = {
25   userId: { type: ObjectId, ref: 'User' }, // reference
26   items: [{ productId: ObjectId, qty: Number, price: Number }],
27   total: Number

```

```
29
30 // Mongoose populate (like $lookup)
31 const orders = await Order.find({ userId })
32   .populate('userId', 'name email') // fetch user fields
33   .populate('items.productId', 'name price image')
```

■ Interview Tip:

There's no single right answer — it depends on your access patterns. Ask: how often is this data read together? How often is it updated? How large can it grow? MongoDB schema design should be driven by your queries, not by normalisation rules.

Q6

What is the \$lookup stage? How to do MongoDB JOINS?

■ Answer

\$lookup performs a left outer join between two collections in the same database. It's MongoDB's equivalent of SQL's LEFT JOIN.

\$lookup syntax:

- from — the other collection to join
- localField — field in current documents
- foreignField — field in the 'from' collection
- as — output array field name

Pipeline \$lookup (advanced):

- Add conditions, filtering, and stages on joined data
- More powerful than simple field match

Performance:

- Always index the foreignField for performance
- \$lookup is expensive — design schemas to minimize need
- Consider embedding if you always join the same data

```

1 // Basic $lookup: get each user's orders
2 const usersWithOrders = await User.aggregate([
3   { $match: { active: true } },
4
5   { $lookup: {
6     from: 'orders',
7     localField: '_id', // User._id
8     foreignField: 'userId', // Order.userId - MUST be indexed!
9     as: 'orders'
10  }},
11
12   // Filter users who have orders
13   { $match: { 'orders.0': { $exists: true } } },
14
15   { $project: {
16     name: 1, email: 1,
17     orderCount: { $size: '$orders' },
18     totalSpent: { $sum: '$orders.total' }
19   }}
20 ])
21
22 // Pipeline $lookup with conditions:
23 await Post.aggregate([
24   { $lookup: {
25     from: 'comments',
26     let: { postId: '$_id' }, // variables from this collection
27     pipeline: [
28       { $match: {

```

```

29       $expr: { $eq: ['$postId', '$$postId'] },

```

```

30     approved: true // only approved comments
31   }},
32   { $sort: { createdAt: -1 } },
33   { $limit: 5 } // only last 5 comments
34 ],
35   as: 'topComments'
36 }}
37 ])
38
39 // Mongoose populate (cleaner for simple lookups)
40 const post = await Post.findById(id)
41   .populate({ path: 'author', select: 'name avatar' })
42   .populate({ path: 'comments', options: { limit: 5, sort: { date: -1 } } })

```

■ Interview Tip:

Always index the foreignField in \$lookup — it's the most important performance optimization for MongoDB JOINS. Without it, every join does a full collection scan on the joined collection.

How to handle transactions in MongoDB?

■ Answer

MongoDB supports multi-document ACID transactions since version 4.0. They're essential for operations that must succeed or fail together.

When to use transactions:

- Transfer money between accounts (debit + credit must both succeed)
- Creating order + reducing inventory (atomic)
- Multi-collection updates that must be consistent

Transaction requirements:

- MongoDB 4.0+ with Replica Set (even single node needs replica set)
- Mongoose 5.2+ for session support

Performance note:

- Transactions are slower than single-document operations
- MongoDB single-document operations are ALWAYS atomic — use when possible
- Design schema to minimize transaction needs (embedding!)

```
1  const mongoose = require('mongoose')
2
3  // Payment transfer with transaction
4  async function transferMoney(fromAccountId, toAccountId, amount) {
5    // Start session
6    const session = await mongoose.startSession()
7
8    try {
9      // Start transaction
10     session.startTransaction()
11
12     // Operation 1: Debit sender
13     const sender = await Account.findOneAndUpdate(
14       { _id: fromAccountId, balance: { $gte: amount } }, // ensure sufficient funds
15       { $inc: { balance: -amount } },
16       { session, new: true } // MUST pass session!
17     )
18
19     if (!sender) {
20       throw new Error('Insufficient funds or account not found')
21     }
22
23     // Operation 2: Credit receiver
24     await Account.findByIdAndUpdate(
25       toAccountId,
26       { $inc: { balance: amount } },
27       { session }
28     )
29
```

```
30 // Operation 3: Record transaction
```

```

31  await Transaction.create([{
32      from: fromAccountId, to: toAccountId,
33      amount, type: 'transfer',
34      status: 'completed'
35  }], { session })
36
37  // Commit – all operations succeed
38  await session.commitTransaction()
39  console.log('Transfer successful')
40
41  } catch(err) {
42      // Rollback – all operations undone!
43      await session.abortTransaction()
44      console.error('Transfer failed, rolled back:', err.message)
45      throw err
46  } finally {
47      session.endSession() // ALWAYS end session!
48  }
49  }

```

■ Interview Tip:

Transactions are essential for fintech apps. At Kanmotech, payment processing requires that both the debit and credit happen atomically — if the credit fails, the debit must be rolled back. Without transactions, you'd have money disappearing.

How to optimize MongoDB queries?

■ Answer

Query optimization is critical for production MongoDB performance.

Optimization checklist:

- Create indexes for all query patterns (check with `explain()`)
- Project only needed fields — don't return the whole document
- Use `$match` early in aggregation pipelines
- Limit result set size — always paginate
- Avoid `$where` and JavaScript operators (slow)
- Use `$in` instead of multiple `$or` conditions
- Compound indexes in correct ESR order

`explain()` — your performance detective:

- `executionStats.totalDocsExamined` — should be close to returned count
- `executionStats.IXSCAN` vs `COLLSCAN` — `IXSCAN` means index used

Common performance killers:

- `COLLSCAN` on large collections
- `$regex` without index (especially without `^`)
- Large `$lookup` without index
- Returning huge documents when only 2 fields needed

```

1 // 1. Use explain() to find query problems
2 const result = await User.find({ email: 'k@example.com' })
3   .explain('executionStats')
4 console.log(result.executionStats.executionStages.stage)
5 // COLLSCAN = problem! IXSCAN = good!
6
7 // 2. Project only needed fields
8 // ■ Returns entire document (may be 50KB)
9 const users = await User.find({ active: true })
10 // ■ Returns only name and email
11 const users2 = await User.find({ active: true }).select('name email -_id')
12
13 // 3. Lean() — skip Mongoose document wrapping (2-3x faster for reads!)
14 const users3 = await User.find().lean() // returns plain JS objects
15 // No .save(), .toJSON(), virtuals — but much faster!
16
17 // 4. Batch operations instead of loops
18 // ■ N queries
19 for (const id of ids) { await Product.findByIdAndUpdate(id, { sale: true }) }
20
21 // ■ 1 query
22 await Product.updateMany({ _id: { $in: ids } }, { $set: { sale: true } })
23
24 // 5. Use cursor for large datasets
25 const cursor = User.find({ active: true }).cursor()

```

```

26 for await (const user of cursor) {

```



```
27   await processUser(user) // processes one at a time – no memory issues!
28 }
29
30 // 6. Compound index for common query pattern
31 // If you always query: { status: 'active', city: 'Mumbai' } ordered by date
32 userSchema.index({ status: 1, city: 1, createdAt: -1 })
```

■ Interview Tip:

`lean()` is one of the biggest easy wins in Mongoose. For read-heavy endpoints that return many documents, adding `.lean()` can be 3-5x faster because Mongoose skips creating full document instances with all their prototype methods.

What is a document in MongoDB?

■ Answer

A document is the basic unit of data in MongoDB, stored as BSON (Binary JSON). It's equivalent to a ROW in SQL databases.

Key characteristics:

- JSON-like structure: key-value pairs
- Can have nested documents and arrays
- Maximum size: 16MB
- Each document has a unique `_id` field
- Documents in the same collection can have DIFFERENT fields

Example document:

- Fields can be: String, Number, Boolean, Date, Array, Object, ObjectId, null

```
1 // MongoDB document (JSON-like)
2 const userDoc = {
3   _id: ObjectId('64abc123...'), // auto-generated unique ID
4   name: 'Kaushal Singh',
5   email: 'k@example.com',
6   age: 27,
7   active: true,
8   createdAt: ISODate('2024-01-15'),
9   // Nested document (embedded object)
10  address: {
11    city: 'Mumbai',
12    state: 'Maharashtra',
13    pincode: '400001'
14  },
15  // Array of strings
16  skills: ['React', 'Node.js', 'MongoDB'],
17  // Array of embedded documents
18  orders: [
19    { orderId: 'ORD001', total: 599, date: ISODate('2024-01-20') },
20    { orderId: 'ORD002', total: 1299, date: ISODate('2024-02-10') }
21  ]
22 }
23
24 // Insert a document
25 await db.collection('users').insertOne(userDoc)
26
27 // Documents in same collection can differ!
28 const adminDoc = {
29   _id: ObjectId('64abc456...'),
30   name: 'Admin User',
31   role: 'superadmin',
32   permissions: ['read', 'write', 'delete']
33   // No email, age, address – that's fine in MongoDB!
34 }
```

■ **Interview Tip:**

Unlike SQL rows, MongoDB documents are flexible — each document can have a completely different structure. This is powerful for polymorphic data (products with different attributes) but requires discipline to maintain consistency.

What is a collection in MongoDB?

■ Answer

A collection is a group of MongoDB documents — equivalent to a TABLE in SQL.

Key differences from SQL tables:

- No fixed schema — documents can have different fields
- No JOINS required — related data can be embedded
- No rows — only documents
- Collections created automatically on first insert

Collections in Mongoose:

- Model name 'User' → collection name 'users' (lowercase + plural)
- Collection name can be set explicitly in schema options

Collection operations:

- `db.createCollection()` — create with options (capped, validator)
- `db.collection.drop()` — delete entire collection
- `db.listCollections()` — list all collections

```

1  const mongoose = require('mongoose')
2
3  // Mongoose auto-creates 'users' collection
4  const User = mongoose.model('User', userSchema)
5
6  // Custom collection name
7  const User2 = mongoose.model('User', userSchema, 'app_users') // → 'app_users'
8
9  // Native driver operations
10 const db = client.db('myapp')
11
12 // List all collections
13 const collections = await db.listCollections().toArray()
14 console.log(collections.map(c => c.name)) // ['users', 'orders', 'products']
15
16 // Create collection with schema validation
17 await db.createCollection('users', {
18   validator: {
19     $jsonSchema: {
20       bsonType: 'object',
21       required: ['name', 'email'],
22       properties: {
23         name: { bsonType: 'string', minLength: 2 },
24         email: { bsonType: 'string', pattern: '^\\S+@\\S+\\.\\.\\S+$' }
25       }
26     }
27   }
28 })
29

```

```

30 // Drop a collection (careful!)

```

```
31 await db.collection('temp_imports').drop()  
32  
33 // Get collection stats  
34 const stats = await db.command({ collStats: 'users' })  
35 console.log(stats.count, stats.size, stats.avgObjSize)
```

■ **Interview Tip:**

Collections are schema-free by default but you can enforce a schema using JSON Schema validators at the database level. This gives you flexibility while maintaining some data integrity without relying entirely on application-level validation.

What is BSON in MongoDB?

■ Answer

BSON (Binary JSON) is the binary-encoded serialization format MongoDB uses to store documents.

BSON vs JSON:

- JSON: text-based, human-readable, limited types
- BSON: binary, faster to traverse, more data types

BSON extra types (not in JSON):

- ObjectId — 12-byte unique identifier
- Date — UTC datetime (not just string)
- Int32, Int64 — precise integer types
- Double — 64-bit floating point
- Decimal128 — high-precision decimals (for money!)
- BinData — binary data (files, images)
- Regex — regular expressions
- Timestamp — internal MongoDB use

Why BSON matters:

- Faster to encode/decode than JSON
- Supports more data types than JSON

```

1 // BSON types in Mongoose schemas
2 const mongoose = require('mongoose')
3
4 const productSchema = new mongoose.Schema({
5   // ObjectId - 12 bytes, auto-indexed
6   categoryId: mongoose.Schema.Types.ObjectId,
7
8   // Date - stored as BSON Date, not string
9   createdAt: { type: Date, default: Date.now },
10
11   // Decimal128 - for money (avoids floating point errors!)
12   price: mongoose.Schema.Types.Decimal128,
13
14   // Buffer - for binary data
15   thumbnail: Buffer,
16
17   // Mixed - any BSON type
18   metadata: mongoose.Schema.Types.Mixed
19 })
20
21 // ObjectId - creating and comparing
22 const { ObjectId } = require('mongodb')
23 const id = new ObjectId() // auto-generated 12-byte ID
24 console.log(id.toString()) // '64abc123def456789012abcd'
25 console.log(id.getTimestamp()) // creation time embedded in ID!
26

```

```

27 // IMPORTANT: compare ObjectIds correctly

```

```
28 const id1 = new ObjectId('64abc123...')
29 const id2 = new ObjectId('64abc123...')
30 console.log(id1 === id2) // false (different objects!)
31 console.log(id1.equals(id2)) // true (correct comparison)
32
33 // In Mongoose: automatic casting
34 await User.findById('64abc123...') // string auto-cast to ObjectId
```

■ Interview Tip:

Always use Decimal128 for monetary values, never JavaScript Number (float). Floating point: $0.1 + 0.2 = 0.30000000000000004$. For a payment system, that's a bug. Decimal128 stores exact decimal values.

What is _id in MongoDB?

■ Answer

_id is the primary key of every MongoDB document — it must be unique within a collection.

Default _id (ObjectId):

- Auto-generated if not provided
- 12 bytes: 4 (timestamp) + 5 (random) + 3 (counter)
- Contains creation timestamp — extract with `getTimestamp()`
- Roughly sortable by creation time

Custom _id:

- Can be any unique value: string, number, UUID
- Useful for natural keys (email, username, order number)

ObjectId advantages:

- Decentralized generation — no DB round trip needed
- Embeds timestamp — free `created_at`
- Sharding-friendly — distributed across shards

```

1 // Auto-generated ObjectId _id (default)
2 const user = await User.create({ name: 'Kaushal', email: 'k@test.com' })
3 console.log(user._id) // ObjectId('64abc123def456789012abcd')
4 console.log(user._id.getTimestamp()) // 2024-01-15T10:30:00.000Z
5
6 // Custom _id — using email as primary key
7 const userSchema = new mongoose.Schema({
8   _id: { type: String }, // custom _id type
9   name: String,
10  role: String
11 })
12 await User.create({ _id: 'kaushal@test.com', name: 'Kaushal', role: 'admin' })
13
14 // Custom _id — using number (serial)
15 const orderSchema = new mongoose.Schema({
16   _id: { type: Number }, // e.g., order number
17   items: Array
18 })
19
20 // Finding by _id
21 const user1 = await User.findById('64abc123...') // string auto-cast
22 const user2 = await User.findOne({ _id: '64abc123...' }) // same
23
24 // ObjectId comparison (IMPORTANT!)
25 const { Types: { ObjectId } } = mongoose
26 user._id.equals(ObjectId('64abc123...')) // correct ■
27 user._id.toString() === '64abc123...' // also works ■
28 user._id === '64abc123...' // WRONG ■
  
```

■ Interview Tip:

Never expose ObjectId as a string in APIs without sanitization. ObjectId embeds a timestamp — from the _id alone you can determine exactly when a user registered. For privacy, prefer UUIDs or hash the ObjectId before exposing it.

What is schema-less in MongoDB?

■ Answer

Schema-less means MongoDB doesn't enforce a fixed structure for documents in a collection. Each document can have different fields, types, and structure.

Benefits:

- Flexibility — add new fields without migrations
- Polymorphism — different shapes in same collection
- Rapid prototyping — no upfront schema design

Risks:

- Inconsistent data — typos become new fields
- Hard to query — which documents have field X?
- Application errors — missing fields cause bugs

Best practice: Schema-less at DB, Schemaful at app

- MongoDB doesn't enforce schema
- Mongoose DOES enforce schema at application level
- Best of both worlds: flexibility + validation

```

1 // Schema-less problem — without Mongoose
2 await db.collection('users').insertMany([
3   { name: 'Kaushal', email: 'k@test.com', age: 27 },
4   { Name: 'Alice', Email: 'a@test.com', Age: '28' }, // different fields!
5   { name: 'Bob', age: -5, email: null } // invalid data!
6 ])
7 // MongoDB accepts ALL of these — no validation!
8
9 // Solution: Mongoose enforces schema at application level
10 const userSchema = new mongoose.Schema({
11   name: { type: String, required: true, trim: true },
12   email: { type: String, required: true, unique: true, lowercase: true },
13   age: { type: Number, min: 0, max: 150 }
14 }, { strict: true }) // strict: true rejects unknown fields
15
16 // Now this throws validation error:
17 await User.create({ Name: 'Alice', Email: 'a@test.com' }) // Error: name required
18
19 // Solution 2: MongoDB JSON Schema Validator
20 await db.createCollection('users', {
21   validator: {
22     $jsonSchema: {
23       required: ['name', 'email'],
24       properties: {
25         name: { bsonType: 'string' },
26         age: { bsonType: 'int', minimum: 0 }
27       },
28       additionalProperties: false // reject unknown fields

```

```

29 }

```

```
30 }  
31 })
```

■ Interview Tip:

MongoDB's schema-less nature is a feature, not a bug — but use it wisely. Use Mongoose strict: true (the default) to prevent typos from creating junk fields. The freedom of schema-less + discipline of Mongoose = best of both worlds.

updateOne vs updateMany in MongoDB?

■ Answer

updateOne updates the FIRST matching document. updateMany updates ALL matching documents.

updateOne:

- Updates first document that matches filter
- Returns: matchedCount, modifiedCount
- Use when: updating a specific document by _id

updateMany:

- Updates ALL documents matching filter
- Returns: matchedCount, modifiedCount
- Use when: bulk updates, status changes

Update operators:

- \$set — set field value
- \$unset — remove a field
- \$inc — increment/decrement a number
- \$push — add to array
- \$pull — remove from array
- \$addToSet — add to array only if not exists

```

1 // updateOne - update specific user
2 const result = await User.updateOne(
3   { _id: userId },           // filter
4   { $set: { name: 'New Name' } } // update
5 )
6 console.log(result.matchedCount) // 1
7 console.log(result.modifiedCount) // 1 (or 0 if no change)
8
9 // updateMany - deactivate all unverified users
10 await User.updateMany(
11   { verified: false, createdAt: { $lt: new Date(Date.now() - 30*24*60*60*1000) } },
12   { $set: { active: false }, $unset: { tempToken: '' } }
13 )
14
15 // Common update operators
16 await User.updateOne({ _id: userId }, {
17   $set: { name: 'Kaushal', 'address.city': 'Pune' }, // set fields
18   $inc: { loginCount: 1, 'stats.visits': 1 },        // increment
19   $push: { tags: 'vip' },                             // add to array
20   $addToSet: { roles: 'editor' },                     // add if not exists
21   $pull: { blockedUsers: someUserId },               // remove from array
22   $unset: { tempToken: '' }                          // remove field
23 })
24
25 // findOneAndUpdate - get document after update
26 const updated = await User.findOneAndUpdate(

```

```

27   userId,

```

```
28 { $inc: { points: 100 } },  
29 { new: true } // return updated doc, not original  
30 )
```

■ Interview Tip:

Always use update operators (\$set, \$inc, etc.) — never replace the whole document accidentally. `updateOne({ _id: id }, { name: 'New' })` would REPLACE the document with just `{ name: 'New' }`! Use `$set: { name: 'New' }` to only update that field.

deleteOne vs deleteMany in MongoDB?

■ Answer

deleteOne removes the FIRST matching document. deleteMany removes ALL matching documents.

deleteOne:

- Removes first document matching filter
- Returns: deletedCount (0 or 1)
- Usually used with _id filter

deleteMany:

- Removes ALL documents matching filter
- Returns: deletedCount (number deleted)
- Use for cleanup, bulk removal

Safe deletion patterns:

- Soft delete — add deleted: true flag instead of removing
- Archive — move to archive collection before deleting
- Always double-check filter before deleteMany!

```

1 // deleteOne — remove specific document
2 const result = await User.deleteOne({ _id: userId })
3 console.log(result.deletedCount) // 1 (deleted) or 0 (not found)
4
5 // Mongoose shorthand
6 await User.findByIdAndDelete(userId) // returns deleted doc
7
8 // deleteMany — remove old sessions
9 const result2 = await Session.deleteMany({
10   expiresAt: { $lt: new Date() } // expired sessions
11 })
12 console.log(`Cleaned up ${result2.deletedCount} sessions`)
13
14 // DANGER: deleteMany with empty filter = delete ALL!
15 // await User.deleteMany({}) // deletes entire collection!
16
17 // Soft delete (preferred for important data)
18 await User.updateOne(
19   { _id: userId },
20   { $set: { deleted: true, deletedAt: new Date() } }
21 )
22 // Then filter all queries:
23 userSchema.pre(/^find/, function(next) {
24   this.find({ deleted: { $ne: true } }) // auto-exclude deleted
25   next()
26 })
27
28 // Archive before deleting
29 const user = await User.findByIdAndDelete(userId)

```

```

30 if (user) {

```

```
31   await ArchivedUser.create({ ...user.toObject(), archivedAt: new Date() })
32 }
```

■ Interview Tip:

Prefer soft delete (marking as deleted) over hard delete for user data. You lose data permanently with hard delete and cannot recover from accidental deletes. Soft delete lets you restore data, audit deletions, and comply with regulations.

Q16

find vs findOne in MongoDB?

■ Answer

find() returns a cursor of all matching documents. findOne() returns the first matching document.

find():

- Returns a cursor (lazy — doesn't fetch until iterated)
- Chain: .sort(), .limit(), .skip(), .select(), .lean()
- Returns empty array [] if no match
- Use for: fetching multiple documents

findOne():

- Returns single document or null
- Internally: find().limit(1)
- Use for: fetching by unique key (email, _id)

findById():

- Mongoose shorthand for findOne({ _id: id })
- Most common for fetching a specific record

```

1 // find() - returns multiple documents
2 const activeUsers = await User.find({ active: true })
3 // Returns array of all active users
4
5 // find() with chaining
6 const topUsers = await User.find({ active: true, role: 'user' })
7   .select('name email avatar') // project fields
8   .sort({ createdAt: -1 }) // newest first
9   .skip(0) // pagination
10  .limit(20) // max 20 results
11  .lean() // plain JS objects (faster reads)
12
13 // find() with no filter - all documents (careful on large collections!)
14 const all = await User.find({})
15
16 // find() returns [] if nothing matches (never null)
17 const none = await User.find({ role: 'superadmin' })
18 console.log(none) // [] - empty array
19
20 // findOne() - single document
21 const user = await User.findOne({ email: 'k@test.com' })
22 console.log(user) // user object or null
23
24 // findById() - Mongoose shorthand (most common)
25 const byId = await User.findById('64abc123...')
26
27 // Null check - findOne returns null if not found
28 if (!user) return res.status(404).json({ error: 'User not found' })
29

```

```

30 // Cursor streaming (large datasets - don't load all in memory!)

```

```
31 const cursor = User.find({ active: true }).cursor()
32 for await (const user of cursor) {
33   await processUser(user) // one at a time
34 }
```

■ Interview Tip:

findOne() is the same as find().limit(1) internally. For fetching by unique key (email, _id), always use findOne or findById — it makes intent clear and MongoDB stops scanning after the first match. Use find() when you expect multiple results.

insertOne vs insertMany in MongoDB?

■ Answer

insertOne inserts a single document. insertMany inserts multiple documents in one network round trip.

insertOne:

- Inserts exactly one document
- Returns: { acknowledged, insertedId }
- Throws on duplicate key (unique index violation)

insertMany:

- Inserts an array of documents in bulk
- Returns: { acknowledged, insertedCount, insertedIds }
- ordered: true (default) — stops on first error
- ordered: false — continues after errors (better throughput)

Performance:

- insertMany is much faster than $N \times$ insertOne
- Use insertMany for bulk imports

```

1 // insertOne - create a single user
2 const result = await User.create({ name: 'Kaushal', email: 'k@test.com' })
3 // OR native driver:
4 const r = await db.collection('users').insertOne({ name: 'Kaushal', email: 'k@test.com' })
5 console.log(r.insertedId) // ObjectId of the new document
6
7 // insertMany - bulk insert
8 const users = [
9   { name: 'Alice', email: 'a@test.com' },
10  { name: 'Bob', email: 'b@test.com' },
11  { name: 'Carol', email: 'c@test.com' },
12 ]
13
14 // Mongoose
15 await User.insertMany(users)
16
17 // Native driver
18 const res = await db.collection('users').insertMany(users)
19 console.log(res.insertedCount) // 3
20
21 // ordered: false - continue even if some fail (e.g. duplicate email)
22 await User.insertMany(users, { ordered: false })
23 // All non-duplicate users inserted, duplicates skipped with errors
24
25 // Bulk import in batches (memory efficient)
26 const allUsers = await readCSV('users.csv') // 100,000 users
27 const BATCH = 500
28 for (let i = 0; i < allUsers.length; i += BATCH) {
29   await User.insertMany(allUsers.slice(i, i + BATCH), { ordered: false })

```

```

30 console.log(`Imported ${Math.min(i + BATCH, allUsers.length)} / ${allUsers.length}`)

```



■ Interview Tip:

For bulk data imports, always use `insertMany` with `ordered: false` and batch size of 500-1000. This gives you the best throughput and doesn't stop on individual duplicate key errors. `ordered: true` stops at the first error, leaving the rest unimported.

What is projection in MongoDB?

■ Answer

Projection specifies which fields to include or exclude from query results. It's like `SELECT column1, column2` in SQL.

Include projection (1):

- Only return specified fields
- `{ name: 1, email: 1 }` — return name and email only
- `_id` included by default unless explicitly excluded

Exclude projection (0):

- Return everything EXCEPT specified fields
- `{ password: 0 }` — return all except password

Cannot mix include and exclude

- `{ name: 1, password: 0 }` is invalid (except `_id`)

Why use projection:

- Reduce network payload (don't transfer 50KB if you need 2 fields)
- Performance — covered queries use index only

```

1 // Include projection – return only specific fields
2 const users = await User.find({ active: true })
3   .select('name email role') // Mongoose syntax
4
5 // MongoDB driver syntax
6 const users2 = await db.collection('users').find(
7   { active: true },
8   { projection: { name: 1, email: 1, role: 1 } }
9 ).toArray()
10
11 // Exclude projection – return all except sensitive fields
12 const users3 = await User.find().select('-password -__v -internalNotes')
13
14 // Exclude _id (included by default)
15 const emails = await User.find().select('email -_id')
16
17 // Nested field projection
18 const partial = await User.find().select('name address.city -_id')
19 // Returns: { name: 'Kaushal', address: { city: 'Mumbai' } }
20
21 // Array slice (projection operator)
22 const post = await Post.findById(id, {
23   title: 1,
24   comments: { $slice: -5 } // only last 5 comments
25 })
26
27 // Always project in production – never return full document

```

```

28 // User document might have 50+ fields, you need 3

```

```
29 const userData = await User.findById(id).select('name email avatar').lean()
```

■ Interview Tip:

Projection is one of the easiest performance wins. If your user document has 50 fields but you only need name and email, you're transferring 48 fields over the network for nothing. Always project, especially in list endpoints returning many documents.

What is a capped collection?

■ Answer

A capped collection is a fixed-size collection that automatically overwrites the oldest documents when full.

Key characteristics:

- Fixed maximum size (in bytes)
- Optional maximum document count
- Automatically deletes oldest documents when full
- Documents inserted in natural order (FIFO)
- Cannot delete individual documents
- Cannot update documents if new size exceeds old size

Use cases:

- Log storage (recent N logs)
- Cache (keep last N items)
- Audit trails (recent activity)
- Real-time analytics buffers

```

1 // Create capped collection
2 await db.createCollection('system_logs', {
3   capped: true,
4   size: 10 * 1024 * 1024, // 10MB max size
5   max: 10000             // max 10,000 documents
6 })
7
8 // Mongoose capped collection
9 const logSchema = new mongoose.Schema({
10   level: String,
11   message: String,
12   service: String,
13   timestamp: { type: Date, default: Date.now }
14 }, {
15   capped: { size: 10485760, max: 10000 }, // 10MB, 10K docs
16   timestamps: false
17 })
18 const Log = mongoose.model('Log', logSchema)
19
20 // Use – logs are automatically purged when full
21 await Log.create({ level: 'error', message: 'DB connection failed', service: 'api' })
22
23 // Tailable cursor – real-time log streaming
24 const cursor = Log.find().tailable({ awaitData: true }).cursor()
25 cursor.on('data', (log) => {
26   console.log(`[${log.level.toUpperCase()}] ${log.message}`)
27 })
28 // Tailable cursor waits for new documents – like tail -f on a log file!
29
30 // Check if collection is capped

```

```

31 const info = await db.collection('system_logs').options()

```

```
32 console.log(info.capped) // true
```

■ Interview Tip:

Capped collections are perfect for logs and activity feeds where you want the last N records. They're much faster than regular collections for this pattern because there's no index maintenance overhead — just a circular buffer.

What is replication in MongoDB?

■ Answer

Replication copies data across multiple MongoDB nodes for high availability and data redundancy.

Why replicate:

- High availability — if primary fails, secondary takes over
- Data redundancy — data safe if one node loses disk
- Read scaling — read from secondaries
- Disaster recovery — replica in another datacenter

How it works:

- Primary receives all writes
- Primary writes to oplog (operations log)
- Secondaries replicate from oplog
- Replication is asynchronous by default

Replication lag:

- Time between write on primary and secondary having it
- Usually milliseconds, can be seconds under load

```

1 // Connect to replica set
2 mongoose.connect('mongodb://node1:27017,node2:27017,node3:27017/mydb', {
3   replicaSet: 'rs0',
4   readPreference: 'secondaryPreferred',
5   writeConcern: { w: 'majority', j: true }
6 })
7
8 // Write concern options:
9 // w: 0 – fire and forget (no confirmation, fastest)
10 // w: 1 – primary confirmed write (default)
11 // w: 'majority' – majority of nodes confirmed (safest)
12
13 // Read preference options:
14 // 'primary' – only read from primary (strong consistency)
15 // 'primaryPreferred' – primary if available, else secondary
16 // 'secondary' – only read from secondary
17 // 'secondaryPreferred' – secondary if available
18 // 'nearest' – lowest network latency
19
20 // Read from secondary (for analytics – may be slightly stale)
21 const recentOrders = await Order.find({ createdAt: { $gte: yesterday } })
22   .read('secondaryPreferred').lean()
23
24 // Monitor replication lag
25 const status = await db.admin().command({ replSetGetStatus: 1 })
26 const lag = status.members
27   .filter(m => m.stateStr === 'SECONDARY')
28   .map(m => ({ host: m.name, lagSec: (Date.now() - m.optimeDate.getTime()) / 1000 })))

```

■ Interview Tip:

Always deploy MongoDB with a 3-node replica set in production — never a single node. A single node has no high availability. With

What is a replica set in MongoDB?

■ Answer

A replica set is a group of MongoDB instances that maintain the same dataset. It provides automatic failover and data redundancy.

Replica set components:

- Primary — receives all write operations
- Secondary — replicates data from primary
- Arbiter — votes in elections, no data (optional)

Minimum config:

- 3 nodes recommended: 1 primary + 2 secondaries
- Or: 2 nodes + 1 arbiter (if budget constrained)
- Always odd number of nodes for clean election

Elections:

- Primary fails → secondaries detect via heartbeat
- Election held — secondary with most up-to-date oplog wins
- New primary elected in ~10-30 seconds
- Requires majority (quorum) of nodes to elect

```

1 // Initialize replica set
2 // Run in mongosh:
3 // rs.initiate({
4 //   _id: 'rs0',
5 //   members: [
6 //     { _id: 0, host: 'mongo1:27017', priority: 2 }, // preferred primary
7 //     { _id: 1, host: 'mongo2:27017', priority: 1 },
8 //     { _id: 2, host: 'mongo3:27017', priority: 1 }
9 //   ]
10 // })
11
12 // Check replica set status
13 const status = await db.admin().command({ replSetGetStatus: 1 })
14 status.members.forEach(m => {
15   console.log(`${m.name}: ${m.stateStr} (health: ${m.health})`)
16   // mongo1: PRIMARY (health: 1)
17   // mongo2: SECONDARY (health: 1)
18   // mongo3: SECONDARY (health: 1)
19 })
20
21 // MongoDB connection string for replica set
22 // mongodb://user:pass@mongo1:27017,mongo2:27017,mongo3:27017/mydb?replicaSet=rs0
23
24 // Mongoose connection
25 mongoose.connect(process.env.MONGO_URI, {
26   replicaSet: 'rs0',
27   w: 'majority',

```

```

28   readPreference: 'secondaryPreferred'

```


■ Interview Tip:

MongoDB Atlas manages replica sets automatically — 3 nodes by default in every tier. For self-hosted, always set priority on the preferred primary node. After a failover, priority determines which node becomes primary when the original comes back online.

Primary vs secondary in MongoDB?

■ Answer

Primary and secondary are roles in a MongoDB replica set.

Primary node:

- Receives ALL write operations
- Only ONE primary at a time
- Records all changes to oplog
- Reads default to primary (strong consistency)

Secondary node:

- Replicates data from primary via oplog
- Can serve reads (set readPreference)
- Cannot accept writes directly
- Participates in elections
- Can become primary if current primary fails

Replication flow:

- Client writes to primary
- Primary records in oplog
- Secondaries poll oplog and apply changes
- Small lag between primary and secondary

```

1 // Write always goes to primary (automatic)
2 await User.create({ name: 'Kaushal' }) // → primary
3
4 // Read from primary (default – strong consistency)
5 const user = await User.findById(id) // → primary
6
7 // Read from secondary (eventual consistency – slightly stale)
8 const users = await User.find({ active: true })
9   .read('secondary')
10 // Useful for: analytics, reporting, heavy read operations
11
12 // Force read from primary even with secondaryPreferred default
13 const criticalData = await Account.findById(accountId)
14   .read('primary') // always fresh
15
16 // Check which node is primary
17 const isMaster = await db.admin().command({ isMaster: 1 })
18 console.log(isMaster.ismaster) // true if this is primary
19 console.log(isMaster.primary) // 'mongo1:27017'
20 console.log(isMaster.hosts) // all replica set members
21
22 // Force election (for maintenance)
23 // rs.stepDown() // current primary steps down, triggers election
  
```

■ Interview Tip:

Never read from secondary for data that requires consistency — like account balances, inventory counts, or auth tokens. The replication lag means secondaries might be milliseconds to seconds behind. Use secondary reads only for analytics and reporting.

What is failover in MongoDB?

■ Answer

Failover is the automatic process of electing a new primary when the current primary becomes unavailable.

Failover process:

1. Primary becomes unavailable (crash, network partition)
2. Secondaries stop receiving heartbeats
3. After ~10 seconds, secondaries call an election
4. Secondary with most up-to-date oplog is elected primary
5. New primary starts accepting writes
6. Total downtime: ~10-30 seconds

During failover:

- Writes fail (no primary available)
- Reads continue from secondaries (if readPreference allows)
- Your application must handle retries

retryWrites=true (MongoDB driver):

- Automatically retries failed writes once after failover
- Works for single-statement operations

```

1 // Handle failover in Node.js
2 mongoose.connect(MONGO_URI, {
3   retryWrites: true,      // retry writes after failover
4   retryReads: true,      // retry reads after failover
5   serverSelectionTimeoutMS: 5000, // fail fast if no server
6   heartbeatFrequencyMS: 2000    // check server health every 2s
7 })
8
9 // Handle connection events
10 mongoose.connection.on('disconnected', () => {
11   console.warn('MongoDB disconnected – failover in progress?')
12 })
13 mongoose.connection.on('reconnected', () => {
14   console.info('MongoDB reconnected – failover complete')
15 })
16
17 // Retry logic for critical operations
18 async function withRetry(operation, maxRetries = 3) {
19   for (let i = 0; i < maxRetries; i++) {
20     try {
21       return await operation()
22     } catch (err) {
23       if (i === maxRetries - 1) throw err
24       if (err.name === 'MongoNetworkError' || err.name === 'MongoServerSelectionError') {
25         console.warn(`Retry ${i + 1}/${maxRetries}...`)
26         await new Promise(r => setTimeout(r, 1000 * (i + 1))) // backoff
27         continue
28       }
29     }
30   }
31 }

```

```
29     throw err // don't retry other errors
30   }
31 }
32 }
33
34 await withRetry(() => Payment.create({ amount: 599 })))
```

■ Interview Tip:

Design your application to handle ~30 seconds of MongoDB unavailability during failover. Implement retry with exponential backoff. Never let a temporary failover cause permanent data loss or user-facing errors.

What is sharding in MongoDB?

■ Answer

Sharding distributes data across multiple MongoDB instances (shards) for horizontal scaling. Each shard holds a subset of the data.

When to shard:

- Data too large for one server (100s of GB)
- Write throughput exceeds single server capacity
- Need geographic data distribution

Sharding components:

- Shards — store subset of data (each is a replica set)
- Mongos — query router (clients connect to this)
- Config servers — store metadata about data distribution

How data is distributed:

- Shard key determines which shard holds each document
- MongoDB splits data into chunks
- Balancer moves chunks between shards

```

1 // Sharding is managed at the MongoDB level, not application
2 // But your app connects to mongos (the router), not individual shards
3
4 mongoose.connect('mongodb://mongos1:27017,mongos2:27017/mydb')
5 // Mongos handles routing to correct shard transparently!
6
7 // Enable sharding (run in mongosh):
8 // sh.enableSharding('mydb')
9 // sh.shardCollection('mydb.users', { userId: 'hashed' }) // hashed shard key
10 // sh.shardCollection('mydb.orders', { customerId: 1, orderId: 1 }) // compound
11
12 // Shard key selection is CRITICAL:
13 // Good shard keys: high cardinality, even distribution, query pattern aligned
14 // Bad shard key: { status: 1 } — only 3 values → hot shards
15
16 // Hashed shard key — even distribution, no range queries
17 // sh.shardCollection('mydb.logs', { _id: 'hashed' })
18
19 // Range shard key — range queries fast, hot shard risk
20 // sh.shardCollection('mydb.orders', { createdAt: 1 })
21
22 // Check sharding status
23 // sh.status() // shows chunks distribution across shards
24
25 // Your queries work the same — mongos routes automatically
26 const user = await User.findById(userId) // mongos → correct shard
  
```

■ Interview Tip:

Don't shard prematurely — it adds enormous operational complexity. Use vertical scaling, then read replicas, then caching. Only shard when you have data > 1TB or write throughput that a single node can't handle.

What is horizontal scaling in MongoDB?

■ Answer

Horizontal scaling (scaling out) adds more servers to distribute load, as opposed to vertical scaling (scaling up) which uses a bigger single server.

MongoDB horizontal scaling options:

- Read replicas — add secondaries, route reads to them
- Sharding — distribute data across multiple primary nodes

Read replica scaling:

- Add secondaries to replica set
- Route read-heavy queries to secondaries
- Scales read throughput linearly

Sharding:

- Distribute data and writes across multiple shards
- Scales both read and write throughput
- Each shard is itself a replica set

```

1 // Step 1: Add read replicas (easiest scaling)
2 // Add secondaries to replica set:
3 // rs.add('mongo4:27017') // new secondary
4
5 // Route reads to secondaries
6 mongoose.connect(MONGO_URI, {
7   readPreference: 'secondaryPreferred' // reads go to replica
8 })
9
10 // Per-query read preference
11 // Heavy analytics query → secondary
12 const stats = await Order.aggregate([
13   { $group: { _id: '$category', total: { $sum: '$amount' } } }
14 ]).read('secondary')
15
16 // Step 2: Sharding (when replicas aren't enough)
17 // Handled at MongoDB level (mongos router)
18 // App connects to mongos and queries work the same
19
20 // MongoDB Atlas auto-scales:
21 // - Vertical: increases instance size
22 // - Storage: auto-expands
23 // - Horizontal: add shards manually (in dedicated tiers)
24
25 // Measure before scaling:
26 const serverStatus = await db.admin().command({ serverStatus: 1 })
27 console.log(serverStatus.connections) // current/available connections
28 console.log(serverStatus.opcounters) // ops/second
29 // If reads > 80% of primary capacity → add replicas

```

```

30 // If writes > 80% of primary capacity → consider sharding

```

■ **Interview Tip:**

Read replicas are the easiest form of horizontal scaling — add secondaries and route reads to them. Only move to sharding when read replicas and vertical scaling aren't enough. Sharding should be a last resort due to its operational complexity.

What is a shard key in MongoDB?

■ Answer

The shard key is the field (or fields) MongoDB uses to distribute documents across shards. Choosing the right shard key is one of the most critical decisions in sharding.

Good shard key properties:

- High cardinality — many unique values (avoid `userId` with 3 values)
- Even distribution — no hot shards
- Query isolation — queries target one shard (not all)

Bad shard key examples:

- `{ status: 1 }` — 3 possible values → all data on 3 shards max
- `{ createdAt: 1 }` — monotonically increasing → all writes to one shard
- `{ country: 1 }` — India has 10x users of others → hot shard

Hashed shard key:

- MongoDB hashes the value — ensures even distribution
- Great for: even writes, no range queries needed
- `{ _id: 'hashed' }` is simple and effective

```

1  // Shard key examples:
2
3  // Good: hashed _id — even distribution
4  // sh.shardCollection('mydb.users', { _id: 'hashed' })
5
6  // Good: compound with high cardinality
7  // sh.shardCollection('mydb.orders', { userId: 1, _id: 1 })
8  // Queries by userId go to specific shard
9
10 // Good: hashed userId — even writes, query isolation
11 // sh.shardCollection('mydb.posts', { userId: 'hashed' })
12
13 // Bad: status — only 3 values
14 // sh.shardCollection('mydb.orders', { status: 1 }) // bad!
15
16 // Bad: createdAt — monotonic, all new docs on same shard
17 // sh.shardCollection('mydb.logs', { createdAt: 1 }) // bad!
18
19 // Check shard key for a collection
20 // db.adminCommand({ shardCollection: 'mydb.users' })
21
22 // Targeted query (efficient — goes to specific shard)
23 await User.findOne({ userId: '123' }) // → one shard
24
25 // Scatter-gather query (expensive — hits ALL shards)
26 await User.find({ age: { $gt: 25 } }) // → all shards (age not in shard key)
27
28 // Rule: always include shard key in your most common queries!

```

■ Interview Tip:

You CANNOT change the shard key after sharding — think carefully before sharding. The shard key is permanent. Many teams

What is chunking in MongoDB sharding?

■ Answer

Chunks are contiguous ranges of shard key values that MongoDB uses to manage data distribution.

How chunks work:

- Data range split into chunks (default 128MB each)
- Each chunk belongs to exactly one shard
- MongoDB balancer moves chunks between shards
- Chunks split automatically when they grow too large

Chunk operations:

- Split — one chunk becomes two (when chunk > 128MB)
- Move — chunk transferred between shards (balancing)
- Merge — empty chunks consolidated

Chunk imbalance:

- If one shard has more chunks → balancer moves chunks
- Target: roughly equal number of chunks per shard

```

1 // Chunks are managed by MongoDB internally
2 // View chunks (in mongosh):
3 // use config
4 // db.chunks.find({ ns: 'mydb.users' }).pretty()
5 // Shows: { min: { userId: MinKey }, max: { userId: '1000' }, shard: 'shard1' }
6
7 // Manual chunk split (advanced – usually not needed)
8 // sh.splitAt('mydb.users', { userId: '500000' })
9
10 // Move chunk manually
11 // db.adminCommand({ moveChunk: 'mydb.users', find: { userId: '123' }, to: 'shard2' })
12
13 // Enable/disable balancer
14 // sh.stopBalancer() // stop during maintenance
15 // sh.startBalancer() // re-enable
16
17 // Check balancer status
18 // sh.getBalancerState() // true/false
19 // sh.isBalancerRunning() // is it currently moving chunks?
20
21 // Application perspective: chunks are invisible
22 // Queries work the same whether data is in 1 or 10 chunks
23 const user = await User.findById(userId) // mongos handles routing
24
25 // Pre-split chunks (before loading data) to avoid hot spots
26 // sh.splitAt('mydb.orders', { createdAt: new Date('2024-01-01') })
27 // sh.splitAt('mydb.orders', { createdAt: new Date('2024-07-01') })

```

■ Interview Tip:

Pre-splitting chunks before bulk data loads prevents the 'jumbo chunk' problem — a single chunk that grows too large to move. For time-series data, pre-split by month before loading historical data.

What is balancing in MongoDB sharding?

■ Answer

The balancer is a MongoDB background process that moves chunks between shards to keep data evenly distributed.

How balancing works:

- Balancer runs on config server
- Periodically checks chunk count per shard
- If imbalance > threshold: migrates chunks to less-loaded shard
- Migration happens in background (no downtime)

Balancer impact:

- Chunk migration consumes I/O and network
- Can slow down normal operations
- Schedule balancer during off-peak hours

When balancer runs too much:

- Bad shard key causing hot spots
- Inserting sequential data (all into one shard)

```
1 // Schedule balancer window (maintenance hours only)
2 // db.adminCommand({
3 //   scheduleBalancerWindow: {
4 //     ns: 'mydb',
5 //     activeWindow: { start: '02:00', stop: '06:00' } // 2 AM to 6 AM
6 //   }
7 // })
8
9 // Stop balancer during bulk import
10 // sh.stopBalancer()
11 // ... bulk import data ...
12 // sh.startBalancer()
13
14 // Check balancer status in mongosh:
15 // sh.status() // shows shard chunk distribution
16 // Example output:
17 // shards:
18 //   shard1  chunks: 10
19 //   shard2  chunks: 9
20 //   shard3  chunks: 11
21 // → balanced! (within 2 chunk difference)
22
23 // Imbalanced example:
24 // shard1  chunks: 50
25 // shard2  chunks: 2
26 // shard3  chunks: 1
27 // → balancer would aggressively migrate chunks!
28
```

```
29 // Your app doesn't need to do anything – balancing is automatic
```

30  // But stop the balancer during maintenance to avoid extra I/O

■ **Interview Tip:**

Stop the balancer during large bulk imports. If you're loading 100M documents and the balancer is trying to migrate chunks at the same time, it creates massive I/O contention and slows both operations significantly.

What is write concern in MongoDB?

■ Answer

Write concern controls how many nodes must acknowledge a write before MongoDB considers it successful.

Write concern levels:

- w: 0 — fire and forget (no acknowledgement, fastest, data loss possible)
- w: 1 — primary acknowledges (default)
- w: 2 — primary + 1 secondary acknowledge
- w: 'majority' — majority of nodes acknowledge (safest)

Journal write concern (j):

- j: true — write flushed to journal (disk) before ack
- j: false — write in memory only (faster, less durable)

wtimeout:

- How long to wait for write concern to be satisfied
- Raises error if not satisfied in time

```

1 // Write concern at connection level
2 mongoose.connect(MONGO_URI, {
3   writeConcern: { w: 'majority', j: true, wtimeout: 5000 }
4 })
5
6 // Write concern per operation
7 await Payment.create([paymentData], {
8   writeConcern: { w: 'majority', j: true } // wait for majority + journal
9 })
10
11 // Fast write (analytics, logs – data loss OK)
12 await EventLog.create([logData], {
13   writeConcern: { w: 0 } // fire and forget
14 })
15
16 // Default (primary confirmed)
17 await Product.findByIdAndUpdate(id, { $inc: { viewCount: 1 } })
18 // writeConcern: { w: 1 } by default
19
20 // Majority with timeout
21 try {
22   await CriticalData.create([data], {
23     writeConcern: { w: 'majority', wtimeout: 3000 }
24   })
25 } catch(err) {
26   if (err.name === 'WriteConcernError') {
27     console.error('Write concern not satisfied – partial replication')
28   }
29 }

```

■ Interview Tip:

For financial data: always use { w: 'majority', j: true }. Yes, it's slower — but if the primary crashes after writing but before replicating to majority, with w:1 you could lose that write permanently. The latency cost is worth the durability guarantee.

What is read concern in MongoDB?

■ Answer

Read concern controls the consistency of data returned from read operations.

Read concern levels:

- 'local' — returns most recent data on this node (default)
- 'available' — like local but for sharded clusters
- 'majority' — data confirmed by majority of nodes
- 'linearizable' — strongest: only after majority confirms
- 'snapshot' — point-in-time consistency (transactions)

When to use majority read concern:

- After majority write — read your own write guarantee
- Financial operations — ensure data not rolled back

Trade-offs:

- 'majority' — slower (waits for majority confirmation)
- 'local' — fastest, may return data that gets rolled back

```

1 // Read concern per query
2 const account = await Account.findById(accountId)
3   .read('primary') // read preference
4   // Read concern set at connection or session level
5
6 // Connection-level read concern
7 mongoose.connect(MONGO_URI, {
8   readConcern: { level: 'majority' }
9 })
10
11 // Per-session (for transactions)
12 const session = await mongoose.startSession({
13   defaultTransactionOptions: {
14     readConcern: { level: 'snapshot' }, // point-in-time read
15     writeConcern: { w: 'majority' }
16   }
17 })
18
19 // Causal consistency — guarantee read-your-own-writes
20 const session2 = await mongoose.startSession()
21 session2.startTransaction()
22 await User.updateOne({ _id: userId }, { $set: { name: 'New Name' } }, { session: session2 })
23 await session2.commitTransaction()
24 // Now read with same session — guaranteed to see your update
25 const user = await User.findById(userId).session(session2)
26 console.log(user.name) // 'New Name' — guaranteed!
27 session2.endSession()

```

■ Interview Tip:

Use readConcern: 'majority' for financial reads to ensure you're reading committed data. If primary fails and gets rolled back, a 'local' read might have returned data that never actually committed permanently.

What is journaling in MongoDB?

■ Answer

Journaling is MongoDB's write-ahead logging mechanism for durability.

Before applying a write to data files, MongoDB records the operation in the journal.

How journaling works:

- Write operation received
- Operation written to in-memory journal buffer
- Journal buffer flushed to disk (every 100ms default)
- Operation applied to data files
- If crash occurs: replay journal on restart

Benefits:

- Crash recovery — journal replayed on restart
- Data durability — writes not lost on crash
- WiredTiger storage engine journals by default

j: true in write concern:

- Write confirmed only after flushed to journal on disk
- Slower but guarantees durability even on crash

```

1 // Journaling is enabled by default in MongoDB
2 // WiredTiger storage engine uses journaling automatically
3
4 // j: true – wait for journal flush before acknowledging
5 await Payment.create(paymentData, {
6   writeConcern: { w: 'majority', j: true }
7   // Write only confirmed after:
8   // 1. Written to majority of nodes' journals
9   // 2. Flushed to disk on those nodes
10  // Even if all nodes crash – data survives!
11 })
12
13 // j: false (or omitted) – faster but less durable
14 await EventLog.create(eventData, {
15   writeConcern: { w: 1, j: false }
16   // Acknowledged when in PRIMARY's MEMORY
17   // If primary crashes before journal flush – data lost!
18 })
19
20 // Check journal status
21 const serverStatus = await db.admin().command({ serverStatus: 1 })
22 console.log(serverStatus.wiredTiger.log) // journal stats
23
24 // Journal commit interval (default 100ms)
25 // mongod --journalCommitInterval=200 // flush every 200ms
26 // Longer interval = better performance, less durability
27

```

```

28 // Disable journaling (not recommended for production!)

```

```
29 // mongod --nojournal // DANGEROUS - only for testing
```

■ **Interview Tip:**

Always use journaling in production. Without journaling, a crash at the wrong moment can corrupt your data files. The performance cost is minimal. The alternative — corrupted or lost data — is catastrophic.

ACID in MongoDB?

■ Answer

MongoDB supports ACID properties at both single-document and multi-document levels.

ACID:

- A — Atomicity: all or nothing
- C — Consistency: data always valid
- I — Isolation: concurrent operations don't interfere
- D — Durability: committed data persists

Single-document ACID (built-in):

- All operations on ONE document are atomic
- No partial updates possible
- Much faster than multi-document transactions

Multi-document ACID (v4.0+):

- Requires replica set
- Use for operations spanning multiple documents/collections
- Similar to SQL transactions

```

1  // ACID single-document — atomic update (no transaction needed)
2  const user = await User.findByIdAndUpdate(
3    userId,
4    {
5      $set: { name: 'New Name', email: 'new@test.com' },
6      $inc: { loginCount: 1 },
7      $push: { loginHistory: { date: new Date(), ip: req.ip } }
8    },
9    { new: true }
10 )
11 // ALL of these changes happen atomically on ONE document
12 // No other operation can see a partial state
13
14 // ACID multi-document — requires transaction
15 const session = await mongoose.startSession()
16 try {
17   await session.withTransaction(async () => {
18     // Atomicity: both must succeed or both fail
19     await Account.findByIdAndUpdate(fromId, { $inc: { balance: -100 } }, { session })
20     await Account.findByIdAndUpdate(toId, { $inc: { balance: 100 } }, { session })
21   })
22 } finally { session.endSession() }
23
24 // Consistency — schema validation ensures valid state
25 // Isolation — transactions use snapshot isolation
26 // Durability — w: 'majority', j: true guarantees persistence

```

■ Interview Tip:

Design your schema to maximize single-document atomicity. Embedding related data means updates are atomic without transactions. Use multi-document transactions only when you truly need cross-document atomicity — they're slower and require replica sets.

What is schema design in MongoDB?

■ Answer

MongoDB schema design is driven by your application's access patterns, not normalization rules.

Design principles:

- Model your data for how you QUERY it, not how it's structured
- Data accessed together should be stored together
- Avoid application-level JOINS when possible

Questions to ask:

- What queries will your app run most often?
- What is the read/write ratio?
- How large can arrays grow?
- How often is related data updated?

Schema design patterns:

- Subset pattern — store frequently used subset inline
- Computed pattern — pre-compute and store aggregated values
- Bucket pattern — group time-series data into buckets
- Outlier pattern — handle celebrity/viral content differently

```

1 // Bad design: following SQL normalization
2 // users, posts, comments, likes – all separate, many JOINS
3
4 // Good design: driven by query patterns
5 // Query: 'Show me a blog post with its recent comments'
6 const postSchema = new mongoose.Schema({
7   title: String,
8   content: String,
9   authorId: ObjectId,           // reference (user updated separately)
10  authorName: String,          // denormalized for fast display
11  authorAvatar: String,        // denormalized
12  tags: [String],              // embedded – always with post
13  likeCount: Number,           // computed pattern – store count
14  commentCount: Number,        // computed pattern
15  recentComments: [{           // subset pattern – last 5 only
16    authorId: ObjectId,
17    authorName: String,
18    text: String,
19    createdAt: Date
20  }],
21 })
22
23 // Bucket pattern – group IoT readings into buckets
24 const sensorBucketSchema = new mongoose.Schema({
25   sensorId: String,
26   date: Date,                 // day bucket
27   count: Number,              // readings in this bucket

```

```

28   measurements: [{

```

```
29     timestamp: Date,  
30     value: Number  
31   }] // up to 200 readings per bucket  
32 })  
33 // Instead of 1M documents (1 per reading) → 5K documents (200 per bucket)
```

■ Interview Tip:

The biggest MongoDB schema design mistake: applying SQL normalization rules. MongoDB rewards denormalization. The question is never 'is this normalized?' but 'does this make my queries fast?' Design for reads first — reads are almost always more frequent than writes.

One-to-many modeling in MongoDB?

■ Answer

One-to-many relationships can be modeled three ways in MongoDB.

Method 1: Embed (one-to-few)

- Store many side inside the one document
- Best for: bounded arrays, always accessed together
- Example: post → comments (max ~100)

Method 2: Reference from many to one

- Many-side documents have a reference to the one
- Best for: large many-side, independently queried
- Example: orders → userId (user has millions of orders)

Method 3: Reference from one to many

- One document stores array of many-side IDs
- Best for: bounded lists, need both sides
- Avoid if array can grow unboundedly

```

1 // One-to-FEW (embed) – post has few comments
2 const postSchema = {
3   title: 'MongoDB Best Practices',
4   comments: [ // embedded – fine for <100
5     { user: 'Alice', text: 'Great post!', date: ISODate() },
6     { user: 'Bob', text: 'Very helpful', date: ISODate() }
7   ]
8 }
9
10 // One-to-MANY (reference on many side) – user has many orders
11 // Order document has userId reference
12 const orderSchema = {
13   _id: ObjectId(),
14   userId: ObjectId('user123'), // reference to user
15   items: [...], total: 599
16 }
17 // Query: all orders for a user
18 const orders = await Order.find({ userId: userId })
19 // Don't store [orderId1, orderId2, ...] on User – unbounded growth!
20
21 // One-to-BOUNDED-MANY (reference on one side) – user has few addresses
22 const userSchema = {
23   name: 'Kaushal',
24   addressIds: [ObjectId('addr1'), ObjectId('addr2')] // bounded list
25 }
26
27 // Two-collection approach with populate
28 const user = await User.findById(userId).populate('addressIds')
29

```

```

30 // Hybrid: count + recent subset

```

```
31 const userWithRecentOrders = {
32   name: 'Kaushal',
33   orderCount: 47,           // denormalized count
34   recentOrderIds: [id1, id2, id3] // last 3 only
35 }
```

■ Interview Tip:

The key question for one-to-many: 'Will the many side grow unboundedly?' If yes (posts → comments), reference from the many side. If bounded (user → addresses), you can embed or reference from the one side. Unbounded embedded arrays kill performance.

Many-to-many modeling in MongoDB?

■ Answer

Many-to-many relationships require careful design in MongoDB.

Common approaches:

- Reference arrays on both sides — each stores IDs of the other
- Junction collection — separate collection for relationships
- Embed subset — most frequent use case embedded

Example: Students ↔ Courses

- Each student in many courses
- Each course has many students

When to use junction collection:

- Relationship has its own attributes (enrollment date, grade)
- One side has very many (10K+ students in course)
- Need to query by relationship attributes

```

1 // Method 1: Reference arrays (small, bounded relationship)
2 const studentSchema = {
3   name: 'Kaushal',
4   courseIds: [ObjectId('cs101'), ObjectId('math201')] // bounded
5 }
6 const courseSchema = {
7   title: 'Computer Science',
8   studentIds: [ObjectId('student1'), ...] // PROBLEM if 10K students!
9 }
10
11 // Method 2: Junction collection (with attributes)
12 const enrollmentSchema = new mongoose.Schema({
13   studentId: { type: ObjectId, ref: 'Student', index: true },
14   courseId: { type: ObjectId, ref: 'Course', index: true },
15   enrolledAt: Date,
16   grade: String,
17   status: { type: String, enum: ['enrolled', 'completed', 'dropped'] }
18 })
19 enrollmentSchema.index({ studentId: 1, courseId: 1 }, { unique: true })
20
21 // Query students in a course
22 const enrollments = await Enrollment.find({ courseId }).populate('studentId', 'name email')
23
24 // Query courses for a student
25 const myCourses = await Enrollment.find({ studentId }).populate('courseId', 'title instructor')
26
27 // Method 3: Embed most relevant data (tags on post)
28 const postSchema = {
29   title: 'MongoDB Deep Dive',
30   tags: ['mongodb', 'database', 'nosql'] // many-to-many via strings

```

```

31 }

```

```
32 // Find all posts with 'mongodb' tag:
33 await Post.find({ tags: 'mongodb' }) // works with multikey index!
```

■ Interview Tip:

Use a junction collection when the relationship has its own attributes (grade, enrollment date, status). Use embedded arrays when the relationship is simple and bounded. Avoid unbounded arrays on either side — they hit the 16MB document limit.

What is denormalization in MongoDB?

■ Answer

Denormalization intentionally duplicates data to improve read performance by avoiding JOINS. It's the opposite of SQL normalization — storing redundant data for speed.

Why denormalize:

- MongoDB doesn't do JOINS efficiently at scale
- Read performance is usually more important than write simplicity
- Pre-computed data avoids expensive aggregations

Common denormalization patterns:

- Embed user name in post (avoid \$lookup on every feed load)
- Store comment count on post (avoid COUNT(*) every page view)
- Duplicate product price in order item (preserve historical price)

Risks:

- Data inconsistency (user changes name — old posts still show old name)
- More storage needed
- Update complexity (must update in multiple places)

```

1 // Denormalized: author info embedded in post
2 const postDoc = {
3   title: 'MongoDB Best Practices',
4   authorId: ObjectId('...'), // reference (source of truth)
5   authorName: 'Kaushal Singh', // DENORMALIZED – duplicated from User
6   authorAvatar: 'https://...', // DENORMALIZED
7   // Trade-off: if user changes name, old posts still show old name
8   // But: fast feed loading (no $lookup needed!)
9 }
10
11 // Denormalized counter pattern
12 const postWithCounts = {
13   title: '...',
14   likeCount: 1547, // denormalized – incremented atomically
15   commentCount: 89, // denormalized
16   shareCount: 23, // denormalized
17   // Avoid: COUNT(*) FROM likes WHERE postId = ? on every page load
18 }
19
20 // Increment atomically (no race condition!)
21 await Post.findByIdAndUpdate(postId, { $inc: { likeCount: 1 } })
22
23 // Price snapshot in order (historical accuracy)
24 const orderItem = {
25   productId: ObjectId('...'),
26   productName: 'iPhone 15 Pro', // denormalized
27   price: 134900, // PRICE AT TIME OF PURCHASE – must never change!
28   quantity: 1
  
```

```

29 }
  
```

```
30 // Even if product price changes to 149900 tomorrow
31 // The order always shows 134900 (correct historical price)
```

■ Interview Tip:

The price-in-order-item is the canonical example of CORRECT denormalization — you WANT the historical snapshot, not the current price. This is intentional. Other denormalization (author name in post) is pragmatic — decide if updating in multiple places is worth the read performance gain.

What is a TTL index in MongoDB?

■ Answer

TTL (Time-To-Live) indexes automatically delete documents after a specified time period. MongoDB runs a background process every 60 seconds to remove expired documents.

Use cases:

- Session management — delete expired sessions
- Cache invalidation — auto-expire cached data
- Audit logs — delete logs older than N days
- OTP/reset tokens — delete used/expired tokens

Requirements:

- Field must be of type Date
- `expireAfterSeconds: 0` means expire at the date stored
- `expireAfterSeconds: 3600` means expire 1 hour after stored date

```

1 // TTL index - expire documents after 1 hour
2 const sessionSchema = new mongoose.Schema({
3   userId: ObjectId,
4   token: String,
5   data: Object,
6   expiresAt: { type: Date, required: true }
7 })
8 // expireAfterSeconds: 0 - expire AT the date stored in expiresAt
9 sessionSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 })
10
11 // Create session that expires in 1 hour
12 await Session.create({
13   userId: user._id,
14   token: 'abc123',
15   expiresAt: new Date(Date.now() + 60 * 60 * 1000) // +1 hour
16 })
17 // MongoDB auto-deletes when Date.now() > expiresAt
18
19 // Alternative: expireAfterSeconds > 0
20 const otpSchema = new mongoose.Schema({
21   email: String,
22   otp: String,
23   createdAt: { type: Date, default: Date.now }
24 })
25 otpSchema.index({ createdAt: 1 }, { expireAfterSeconds: 600 }) // expire 10 min after creation
26
27 // Note: TTL deletion runs every 60 seconds, not exactly at expiry time
28 // Documents may live up to 60 seconds past expiry
29
30 // Update expiry (extend session)
31 await Session.findByIdAndUpdate(sessionId, {
32   $set: { expiresAt: new Date(Date.now() + 60 * 60 * 1000) } // extend 1 hour
33 })
  
```

■ **Interview Tip:**

TTL indexes are perfect for session management. Instead of writing a cron job to delete expired sessions, create a TTL index and MongoDB handles it automatically. The 60-second cleanup granularity is fine for most use cases.

What is a partial index in MongoDB?

■ Answer

A partial index only indexes documents that match a specified filter expression. Smaller than a full index — only indexes the documents you actually query.

Benefits:

- Smaller index size — less memory and storage
- Faster index writes — fewer documents indexed
- Better performance than full index for filtered queries

Use cases:

- Index only active users (not all users)
- Index only pending orders (not completed/cancelled)
- Index only high-value transactions

Partial unique index:

- Enforce uniqueness only for matching documents
- Useful: unique email only for verified users

```

1 // Full index — indexes ALL products (even discontinued)
2 productSchema.index({ category: 1 })
3
4 // Partial index — only index ACTIVE products
5 productSchema.index(
6   { category: 1, price: -1 },
7   { partialFilterExpression: { active: true } } // only index active products
8 )
9 // When you query { category: 'Electronics', active: true }
10 // MongoDB uses this smaller, faster index
11 // If you query without { active: true }, falls back to COLLSCAN
12
13 // Partial unique index — unique email only for verified accounts
14 userSchema.index(
15   { email: 1 },
16   {
17     unique: true,
18     partialFilterExpression: { verified: true } // only verified users
19   }
20 )
21 // Now: two unverified users can have same email (pending verification)
22 // But: verified users must have unique emails
23
24 // Partial index for pending orders only
25 ordersSchema.index(
26   { createdAt: -1 },
27   { partialFilterExpression: { status: 'pending' } }
28 )
29 // Only pending orders are indexed — much smaller than all orders

```

```

30 // Query: await Order.find({ status: 'pending' }).sort({ createdAt: -1 })

```

■ **Interview Tip:**

Partial indexes are underused but very powerful. If 95% of your queries filter by status: 'active', and only 10% of documents are active, your partial index is 10x smaller and 10x faster than a full index. Always use partial indexes for filtered collections.

What is a sparse index in MongoDB?

■ Answer

A sparse index only indexes documents that have the indexed field. Documents without the field are excluded from the index.

Sparse vs regular index:

- Regular: indexes ALL documents (with null for missing field)
- Sparse: skips documents where field doesn't exist

Use cases:

- Optional fields that only some documents have
- Unique constraint on an optional field
- Avoid null entries in index

Sparse unique index:

- Multiple documents can have null/missing field
- But documents WITH the field must have unique values

```

1 // Sparse index – only index documents with 'phone' field
2 userSchema.index({ phone: 1 }, { sparse: true })
3 // Documents without 'phone' are NOT in this index
4 // Much smaller index if most users don't have phone
5
6 // Sparse unique index – optional unique field
7 userSchema.index({ githubId: 1 }, { unique: true, sparse: true })
8 // Users without githubId: multiple allowed (not indexed)
9 // Users with githubId: must be unique
10 // Perfect for OAuth providers!
11
12 // Without sparse: regular unique index on optional field
13 // userSchema.index({ githubId: 1 }, { unique: true })
14 // Problem: only ONE user can have null githubId!
15 // All users without githubId would conflict on unique null value
16
17 // Partial index is usually better than sparse in MongoDB 3.2+
18 // Partial: you control exactly which documents are indexed
19 userSchema.index(
20   { githubId: 1 },
21   { unique: true, partialFilterExpression: { githubId: { $exists: true } } }
22 )
23 // More explicit than sparse: true
24
25 // Check if index is sparse
26 const indexes = await User.collection.getIndexes()
27 console.log(indexes) // shows { sparse: true } for sparse indexes

```

■ Interview Tip:

Use sparse indexes for optional fields that only some documents have. Without sparse, a unique index allows only ONE null value — so only one user can have a missing field. Sparse allows unlimited documents with missing field while enforcing uniqueness for those that have it.

What is collation in MongoDB?

■ Answer

Collation specifies language-specific rules for string comparison, like case and accent sensitivity.

Use cases:

- Case-insensitive queries and sorting
- Language-specific sorting (German ä after a or z?)
- Accent-insensitive search

Key collation options:

- locale — language (en, fr, de, hi)
- strength — comparison level (1=base, 2+=accents, 3+=case)
- caseLevel — enable case comparison at strength 1/2

Strength levels:

- 1 — base characters only (a=A=à)
- 2 — base + accents (a≠à, a=A)
- 3 — base + accents + case (a≠A, a≠à) [default]

```

1 // Case-insensitive index
2 userSchema.index(
3   { name: 1 },
4   { collation: { locale: 'en', strength: 2 } } // ignore case
5 )
6
7 // Case-insensitive query (must use same collation as index!)
8 const user = await User.findOne({ name: 'kaushal' }).collation({ locale: 'en', strength: 2 })
9 // Finds: 'Kaushal', 'KAUSHAL', 'kaushal' — all match
10
11 // Case-insensitive unique email (better than .toLowerCase())
12 userSchema.index(
13   { email: 1 },
14   { unique: true, collation: { locale: 'en', strength: 2 } }
15 )
16 // 'user@test.com' and 'User@Test.com' are the same — only one allowed
17
18 // Sorted by name (locale-aware sorting)
19 const users = await User.find().sort({ name: 1 }).collation({ locale: 'en' })
20 // Sorts: 'apple', 'Banana', 'cherry' (not 'Banana', 'apple', 'cherry')
21 // Without collation, uppercase sorts before lowercase (ASCII order)
22
23 // Collection-level collation
24 await db.createCollection('users', {
25   collation: { locale: 'en', strength: 2 } // all queries use this
26 })

```

■ Interview Tip:

Collation strength 2 is the sweet spot for most apps — case-insensitive but accent-sensitive. Strength 1 treats 'naïve' and 'naive' as equal. If you store emails, always use collation or normalize to lowercase before storage.

What is profiling in MongoDB?

■ Answer

MongoDB profiler captures information about database operations for performance analysis. Similar to slow query log in MySQL.

Profiling levels:

- 0 — off (default)
- 1 — slow queries only (above slowms threshold)
- 2 — all queries (HIGH OVERHEAD — development only!)

What profiler captures:

- Query shape, indexes used, execution time
- Documents examined vs returned
- Response time

Storage:

- Saved to system.profile capped collection
- Default 1MB size (limited history)

```

1 // Enable profiling (production: level 1, slowms: 100ms)
2 await db.setProfilingLevel(1, { slowms: 100 }) // log queries > 100ms
3
4 // Enable all queries (DEVELOPMENT ONLY!)
5 await db.setProfilingLevel(2) // captures EVERYTHING – huge overhead!
6
7 // View slow queries
8 const slowQueries = await db.collection('system.profile')
9   .find({ millis: { $gt: 100 } })
10  .sort({ ts: -1 })
11  .limit(10)
12  .toArray()
13
14 slowQueries.forEach(q => {
15   console.log({
16     operation: q.op,           // 'query', 'update', etc.
17     namespace: q.ns,          // 'mydb.users'
18     duration: q.millis,        // milliseconds
19     docsExamined: q.docsExamined, // documents scanned
20     docsReturned: q.nreturned,  // documents returned
21     plan: q.execStats?.stage,    // IXSCAN or COLLSCAN
22     filter: q.query || q.command?.filter
23   })
24 })
25
26 // Find most frequent slow queries
27 await db.collection('system.profile').aggregate([
28   { $group: { _id: '$query', count: { $sum: 1 }, avgMs: { $avg: '$millis' } } },
29   { $sort: { count: -1 } },

```

```

30 { $limit: 10 }

```

```
31 ])  
32  
33 // Get current profiling level  
34 const level = await db.getProfilingLevel()  
35 console.log(level) // { was: 1, slowms: 100 }
```

■ Interview Tip:

Set profiling level 1 with slowms: 100 in production — you only capture queries slower than 100ms. Level 2 in production captures EVERY query and can seriously impact performance. Use level 2 only in development or staging.

How to design a MongoDB schema?

■ Answer

Schema design is the most important MongoDB skill — get it right and queries are fast.

Design process:

1. List all queries your app will run
2. Note access patterns: read vs write ratio
3. Group data that's queried together
4. Decide embed vs reference per relationship
5. Create indexes for all query patterns
6. Test with realistic data volume

Golden rules:

- Design for your queries, not normalization
- Data queried together lives together
- Avoid unbounded arrays
- Pre-compute what you can (counts, aggregates)

```

1 // Example: E-commerce schema design process
2
3 // Step 1: List queries
4 // Q1: Get product with details (most frequent - product page)
5 // Q2: Get user's order history (frequent - my orders)
6 // Q3: Get order with line items and user info (moderate)
7 // Q4: Get products by category with filters (frequent - catalog)
8 // Q5: Analytics: daily revenue by category (rare - admin)
9
10 // Step 2: Design schema around these queries
11 const productSchema = {
12   _id: ObjectId,
13   name: String, price: Number,
14   category: { id: ObjectId, name: String }, // denormalized for Q4
15   images: [String],
16   // embedded – always with product
17   specs: Object, // embedded – always with product (flexible attrs)
18   rating: { avg: Number, count: Number }, // computed – avoid aggregation
19   inStock: Boolean
20 }
21
22 const orderSchema = {
23   _id: ObjectId,
24   userId: ObjectId, // reference (Q2 filter field – indexed!)
25   status: String,
26   createdAt: Date,
27   items: [{ // embedded – always needed with order
28     productId: ObjectId,
29     name: String, // denormalized – historical snapshot
30     price: Number, // snapshot at time of purchase
31     quantity: Number
32   }],

```

```
33   total: Number
34 }
35
36 // Step 3: Indexes for all queries
37 productSchema.index({ category: 1, price: 1 }) // Q4
38 productSchema.index({ name: 'text' })         // search
39 orderSchema.index({ userId: 1, createdAt: -1 }) // Q2
40 orderSchema.index({ status: 1, createdAt: -1 }) // admin
```

■ Interview Tip:

Schema design is an iterative process. Start simple, measure query performance with real data, then optimize. The biggest mistake is designing for hypothetical future queries instead of actual current needs. Design for what your app does today.

Pagination best practices in MongoDB?

■ Answer

Efficient pagination is critical for collections with millions of documents.

Two pagination methods:

- Skip/Limit — simple but slow on large offsets
- Range/Cursor — efficient, consistent results

Skip/Limit problems:

- skip(10000) must read and discard 10000 documents
- Gets slower as page number increases
- Inconsistent if documents added/removed

Range query (cursor pagination):

- Use lastId from previous page as cursor
- Always $O(\log n)$ — no skip cost
- Consistent — no duplicates/skips on inserts

When to use each:

- Skip/limit: < 1000 documents, user-facing pages
- Range pagination: large datasets, APIs, infinite scroll

```

1 // Skip/limit — simple but slow on deep pages
2 app.get('/products', async (req, res) => {
3   const page = parseInt(req.query.page) || 1
4   const limit = 20
5   const skip = (page - 1) * limit
6   const products = await Product.find({ active: true })
7     .sort({ createdAt: -1 })
8     .skip(skip) // ■ page 1000 = skip 19,980 docs!
9     .limit(limit)
10    .lean()
11   res.json(products)
12 })
13
14 // Range pagination — always fast!
15 app.get('/products/cursor', async (req, res) => {
16   const { cursor, limit = 20 } = req.query
17   const query = { active: true }
18   if (cursor) {
19     query._id = { $lt: new mongoose.Types.ObjectId(cursor) } // after last item
20   }
21   const products = await Product.find(query)
22     .sort({ _id: -1 }) // newest first
23     .limit(parseInt(limit) + 1) // fetch 1 extra to check if more exist
24     .lean()
25   const hasMore = products.length > limit
26   if (hasMore) products.pop() // remove the extra

```

```

27   const nextCursor = hasMore ? products[products.length - 1]._id : null

```

```
28   res.json({ data: products, nextCursor, hasMore })
29 })
30
31 // $facet for page + total count in one query
32 const result = await Product.aggregate([
33   { $match: { active: true } },
34   { $facet: {
35     data: [{ $skip: skip }, { $limit: limit }],
36     total: [{ $count: 'count' }]
37   }}
38 ])
```

■ Interview Tip:

Use cursor-based pagination for any list that might have thousands of items. Apps like Twitter, Instagram, and Slack all use cursor pagination. `skip(10000)` is $O(N)$ — it literally reads and discards 10,000 documents. Cursor pagination is always $O(\log N)$.

Skip vs range query in MongoDB?

■ Answer

Skip and range queries are two fundamentally different pagination approaches.

Skip-based:

- `db.find().skip(N).limit(20)`
- Simple: $\text{page} = N/20 + 1$
- Problem: must scan N documents to skip them
- Gets progressively slower: page 100 scans 2000 docs

Range-based (cursor):

- `db.find({ _id: { $gt: lastId } }).limit(20)`
- Uses index directly — $O(\log n)$
- Consistent: new inserts don't affect pagination

Performance comparison:

- Page 1 (skip 0): both equal
- Page 100 (skip 2000): skip is 100x slower
- Page 1000 (skip 20000): skip is 1000x slower

```

1 // Skip-based: works fine for small datasets or early pages
2 const page1 = await Product.find().sort({ _id: -1 }).skip(0).limit(20) // fast
3 const page10 = await Product.find().sort({ _id: -1 }).skip(180).limit(20) // ok
4 const page100 = await Product.find().sort({ _id: -1 }).skip(1980).limit(20) // slow!
5 const page1000 = await Product.find().sort({ _id: -1 }).skip(19980).limit(20) // very slow!
6 // Each skip must scan ALL documents before the skip point
7
8 // Range-based: always uses index, always fast
9 // First page
10 const firstPage = await Product.find().sort({ _id: -1 }).limit(20).lean()
11 const lastId = firstPage[firstPage.length - 1]._id
12
13 // Next page – using lastId as cursor
14 const nextPage = await Product.find({ _id: { $lt: lastId } })
15   .sort({ _id: -1 })
16   .limit(20)
17   .lean()
18 // MongoDB jumps directly to lastId using index – no scanning!
19
20 // Compound cursor for non-_id sort
21 // Sort by price, then _id for stability
22 productSchema.index({ price: -1, _id: -1 })
23 const page = await Product.find({
24   $or: [
25     { price: { $lt: lastPrice } },
26     { price: lastPrice, _id: { $lt: lastId } } // tiebreaker
27   ]
28 }).sort({ price: -1, _id: -1 }).limit(20)

```

■ Interview Tip:

Range query is $O(\log N)$ because it uses an index lookup, while skip is $O(N)$ because it must scan N documents. For a collection

How to handle large datasets in MongoDB?

■ Answer

Large datasets (100M+ documents) require specific strategies to maintain performance.

Strategies:

- Indexes — create for ALL query patterns
- Projections — return only needed fields
- Cursor-based pagination — avoid skip on large collections
- Streaming — process with cursors instead of loading all in memory
- Archiving — move old data to separate collection
- Sharding — distribute across multiple servers

Working set:

- MongoDB keeps hot data in RAM (working set)
- If working set > RAM, queries hit disk (slow!)
- Solution: ensure indexes + active data fit in RAM

```

1 // 1. Stream large datasets instead of loading all into memory
2 const cursor = Order.find({ status: 'completed' })
3   .select('userId total createdAt') // only needed fields
4   .sort({ createdAt: -1 })
5   .cursor() // stream results
6
7 for await (const order of cursor) {
8   await processOrder(order) // process one at a time
9 }
10 // Never loads all orders into memory!
11
12 // 2. Batch processing large updates
13 async function archiveOldOrders() {
14   const batchSize = 1000
15   const cutoff = new Date(Date.now() - 365 * 24 * 60 * 60 * 1000)
16   let lastId = null
17
18   while (true) {
19     const query = { createdAt: { $lt: cutoff } }
20     if (lastId) query._id = { $gt: lastId }
21     const batch = await Order.find(query).limit(batchSize).lean()
22     if (!batch.length) break
23
24     await ArchiveOrder.insertMany(batch, { ordered: false })
25     await Order.deleteMany({ _id: { $in: batch.map(o => o._id) } })
26     lastId = batch[batch.length - 1]._id
27     await new Promise(r => setTimeout(r, 100)) // brief pause - don't overwhelm
28   }
29 }
30
31 // 3. Time-series collections (MongoDB 5.0+)

```

```

32 // Optimized for time-series data, 90% storage reduction!

```

```
33 await db.createCollection('sensor_readings', {  
34   timeseries: { timeField: 'timestamp', metaField: 'sensorId', granularity: 'seconds' }  
35 })
```

■ Interview Tip:

The working set is the key concept for MongoDB performance. Keep your indexes + frequently accessed documents in RAM. If your dataset is 500GB but your queries only touch the last 30 days (10GB), ensure RAM is at least 10GB. Query patterns determine the working set size.

Caching with MongoDB?

■ Answer

Caching strategies with MongoDB reduce database load and improve response times.

Caching options:

- Redis — external cache (most common)
- Application memory — in-process cache (fastest, not shared)
- MongoDB itself — not a cache, but can cache results

What to cache:

- Frequently read, rarely changed: product catalog
- Expensive aggregations: analytics dashboards
- User session data

Cache invalidation:

- TTL — expire after N seconds
- Event-driven — delete cache on update
- Version-based — cache:v2:product:123

```
1  const redis = require('ioredis').createClient()
2
3  // Cache-aside pattern with MongoDB + Redis
4  async function getProduct(productId) {
5      const cacheKey = `product:${productId}`
6
7      // Check Redis first
8      const cached = await redis.get(cacheKey)
9      if (cached) return JSON.parse(cached)
10
11     // Cache miss: query MongoDB
12     const product = await Product.findById(productId).lean()
13     if (!product) return null
14
15     // Store in Redis with TTL
16     await redis.setex(cacheKey, 3600, JSON.stringify(product)) // 1 hour
17     return product
18 }
19
20 // Invalidate cache on update
21 async function updateProduct(productId, changes) {
22     const updated = await Product.findByIdAndUpdate(productId, changes, { new: true })
23     await redis.del(`product:${productId}`) // invalidate!
24     return updated
25 }
26
27 // Cache aggregation results
28 async function getCategoryStats() {
29     const cacheKey = 'category:stats'
```

```
30     const cached = await redis.get(cacheKey)
```



```
31  if (cached) return JSON.parse(cached)
32
33  const stats = await Order.aggregate([
34    { $group: { _id: '$category', total: { $sum: '$amount' } } }
35  ])
36
37  await redis.setex(cacheKey, 300, JSON.stringify(stats)) // 5 min cache
38  return stats
39 }
```

■ Interview Tip:

MongoDB has an internal WiredTiger cache (50% of RAM by default) that caches frequently accessed data. But for application-level caching (reducing MongoDB queries), always use Redis. The WiredTiger cache is internal — you don't control what it caches.

Data archiving in MongoDB?

■ Answer

Data archiving moves old or rarely accessed data to cheaper, slower storage to improve performance and reduce costs.

Archiving strategies:

- Separate collection — same MongoDB, different collection
- Separate database — different MongoDB database
- Different cluster — cheaper MongoDB Atlas tier
- Object storage — S3/GCS (no longer queryable with MQL)

Benefits:

- Smaller working set — more frequently accessed data in RAM
- Better query performance on active data
- Cost reduction — cheaper storage for old data

Online Archive (MongoDB Atlas):

- Automatically archives data to S3 based on date
- Still queryable via mongos (federated)

```

1 // Strategy 1: Separate archive collection
2 async function archiveOldOrders(daysOld = 365) {
3   const cutoffDate = new Date(Date.now() - daysOld * 24 * 60 * 60 * 1000)
4
5   // Batch process to avoid memory issues
6   const batchSize = 500
7   while (true) {
8     const oldOrders = await Order
9       .find({ createdAt: { $lt: cutoffDate }, archived: { $ne: true } })
10      .limit(batchSize)
11      .lean()
12
13     if (!oldOrders.length) break
14
15     // Copy to archive collection
16     await ArchiveOrder.insertMany(
17       oldOrders.map(o => ({ ...o, archivedAt: new Date() })),
18       { ordered: false }
19     )
20
21     // Mark as archived (or delete from active)
22     await Order.updateMany(
23       { _id: { $in: oldOrders.map(o => o._id) } },
24       { $set: { archived: true } }
25     )
26   }
27   console.log('Archiving complete')
28 }

```

29

```
30 // Query active vs archived
31 const activeOrders = await Order.find({ archived: { $ne: true } })
32 const archivedOrders = await ArchiveOrder.find({ createdAt: { $lt: cutoffDate } })
33
34 // Scheduled archiving with cron
35 const cron = require('node-cron')
36 cron.schedule('0 2 * * *', async () => { // 2 AM daily
37   console.log('Running archiving job')
38   await archiveOldOrders(365) // archive orders > 1 year
39 })
```

■ Interview Tip:

Run archiving jobs at low-traffic hours (2-3 AM) in small batches. Archiving 1M documents in one shot can overload your system. Batch size of 500 with a small delay between batches keeps the load manageable.

Scaling MongoDB?

■ Answer

MongoDB scales both vertically (bigger machine) and horizontally (more machines).

Scaling steps (in order):

1. Optimize queries + indexes (free!)
2. Add more RAM (working set optimization)
3. Upgrade to faster storage (NVMe SSD)
4. Add read replicas (read scaling)
5. Shard the database (write + data scaling)

Working set = key metric:

- Working set: indexes + frequently accessed data
- Must fit in RAM for best performance
- If working set > RAM → disk I/O → 100x slower

Atlas auto-scaling:

- Vertical: auto-scales compute and memory
- Storage: auto-expands when needed

```

1 // Step 1: Monitor before scaling
2 const status = await db.admin().command({ serverStatus: 1 })
3 console.log({
4   connections: status.connections,           // current/available
5   opsPerSec: status.opcounters,              // reads/writes per second
6   cacheHitRatio: status.wiredTiger.cache['pages read into cache'] /
7                 status.wiredTiger.cache['pages read from cache'],
8   memorySizeMB: status.mem.resident,         // RAM usage
9 })
10 // cache hit < 0.9 → working set doesn't fit in RAM → need more RAM!
11
12 // Step 2: Index optimization
13 const missing = await db.collection('orders').aggregate([
14   { $indexStats: {} }
15 ]).toArray()
16 // Find indexes never used → drop them (waste of memory)
17
18 // Step 3: Add read replica
19 // rs.add('mongo4:27017') // new secondary
20 mongoose.connect(uri, { readPreference: 'secondaryPreferred' })
21
22 // Step 4: Connection pooling (handle more concurrent requests)
23 mongoose.connect(uri, { maxPoolSize: 50, minPoolSize: 10 })
24
25 // Step 5: Shard (last resort)
26 // sh.enableSharding('mydb')
27 // sh.shardCollection('mydb.orders', { userId: 'hashed' })
28 // Now connects to mongos, not primary directly

```

■ Interview Tip:

The most common scaling mistake: jumping to sharding before exhausting simpler options. Add indexes, increase RAM, add read

Monitoring MongoDB?

■ Answer

Monitoring is essential for maintaining MongoDB performance and health in production.

Key metrics to monitor:

- Operation rates — reads, writes, commands per second
- Connections — current, available, max
- Cache hit ratio — WiredTiger cache effectiveness
- Replication lag — secondary falling behind
- Query performance — slow queries from profiler

Monitoring tools:

- MongoDB Atlas — built-in monitoring dashboard
- MongoDB Ops Manager — self-hosted management
- mongostat — command-line real-time stats
- mongotop — collection-level operation stats
- Datadog / New Relic — third-party APM

```

1 // serverStatus — comprehensive metrics
2 const status = await db.admin().command({ serverStatus: 1 })
3 console.log({
4   // Connections
5   currentConnections: status.connections.current,
6   availableConnections: status.connections.available,
7
8   // Operations
9   queriesPerSec: status.opcounters.query,
10  insertsPerSec: status.opcounters.insert,
11  updatesPerSec: status.opcounters.update,
12
13  // WiredTiger cache hit ratio
14  cacheHitRatio: (
15    status.wiredTiger.cache['pages read into cache'] /
16    status.wiredTiger.cache['pages requested from the cache']
17  ).toFixed(3), // should be > 0.99
18
19  // Memory
20  residentMemoryMB: status.mem.resident,
21  virtualMemoryMB: status.mem.virtual
22 })
23
24 // Replica set monitoring
25 const replStatus = await db.admin().command({ replSetGetStatus: 1 })
26 const primaryOptime = replStatus.members.find(m => m.stateStr === 'PRIMARY').optime
27 replStatus.members.filter(m => m.stateStr === 'SECONDARY').forEach(secondary => {
28   const lagMs = primaryOptime.ts.getTime() - secondary.optime.ts.getTime()
29   console.log(`${secondary.name} lag: ${lagMs}ms`)
30   if (lagMs > 5000) console.warn(`HIGH REPLICATION LAG: ${secondary.name}!`)
31 })

```

```
32
33 // Prometheus metrics for Grafana dashboards
34 // npm install mongodb-exporter
35 // Exposes metrics at /metrics for Prometheus scraping
```

■ **Interview Tip:**

Cache hit ratio is the most important MongoDB performance metric. If < 0.95 , your working set doesn't fit in RAM and MongoDB is hitting disk for most queries. This is 100x slower than RAM access. Fix: add RAM, reduce working set, or archive old data.

Metrics to track in MongoDB?

■ Answer

These metrics tell you if MongoDB is healthy and performing well.

Critical metrics:

- Cache hit ratio > 0.95 (working set fits in RAM)
- Replication lag < 10 seconds
- Connections used < 80% of max
- No COLLSCAN on frequently run queries
- Slow queries < 5% of total operations

Performance metrics:

- Reads/sec, Writes/sec, Commands/sec
- Avg query duration
- Document fetch rate vs document scan rate

Capacity metrics:

- Disk usage and growth rate
- RAM usage vs total RAM
- Oplog window (hours of replication buffer)

```

1 // Key metrics collection
2 async function collectMetrics() {
3   const [serverStatus, dbStats, replStatus] = await Promise.all([
4     db.admin().command({ serverStatus: 1 }),
5     db.stats(),
6     db.admin().command({ replSetGetStatus: 1 }).catch(() => null)
7   ])
8
9   const metrics = {
10     // Throughput
11     reads_per_sec: serverStatus.opcounters.query,
12     writes_per_sec: serverStatus.opcounters.insert + serverStatus.opcounters.update + serverStatus.opcounters.merge,
13
14     // Cache
15     cache_hit_ratio: (serverStatus.wiredTiger.cache['pages read into cache'] /
16       serverStatus.wiredTiger.cache['pages requested from the cache']),
17
18     // Connections
19     connections_used: serverStatus.connections.current,
20     connections_available: serverStatus.connections.available,
21
22     // Storage
23     data_size_gb: dbStats.dataSize / (1024**3),
24     storage_size_gb: dbStats.storageSize / (1024**3),
25
26     // Memory
27     resident_memory_mb: serverStatus.mem.resident,
28   }
  
```



```
29
30 // Replication lag
31 if (replStatus) {
32   const primary = replStatus.members.find(m => m.stateStr === 'PRIMARY')
33   replStatus.members.filter(m => m.stateStr === 'SECONDARY').forEach(sec => {
34     metrics[`replication_lag_${sec.name.replace(':', '_')}`] =
35       (primary.optime.ts.getTime() - sec.optime.ts.getTime()) / 1000
36   })
37 }
38 return metrics
39 }
40
41 // Send to Datadog/CloudWatch every 30 seconds
42 setInterval(async () => {
43   const metrics = await collectMetrics()
44   await sendToMonitoring(metrics)
45 }, 30000)
```

■ Interview Tip:

Set up alerts on cache hit ratio < 0.90 and replication lag > 30s — these are the two leading indicators of MongoDB performance problems before they become user-visible incidents.

Slow query log in MongoDB?

■ Answer

MongoDB's profiler captures slow queries for performance analysis.

Configuration:

- Level 0: off
- Level 1: log queries > slowms threshold
- Level 2: log all queries (never in production!)

Analysis process:

1. Enable profiler with appropriate slowms threshold
2. Let application run normally
3. Query system.profile for patterns
4. Identify top offenders by frequency × duration
5. Add indexes or optimize queries

Atlas slow query log:

- Available in Performance Advisor tab
- Automatically suggests missing indexes

```

1 // Enable slow query logging (production-safe)
2 await db.setProfilingLevel(1, { slowms: 100 }) // log queries > 100ms
3
4 // Find the worst queries
5 const slowQueries = await db.collection('system.profile').aggregate([
6   // Only look at recent queries
7   { $match: { ts: { $gte: new Date(Date.now() - 24 * 60 * 60 * 1000) } } },
8
9   // Group by query shape
10  { $group: {
11    _id: { op: '$op', ns: '$ns', query: '$query' },
12    count: { $sum: 1 },
13    avgMs: { $avg: '$millis' },
14    maxMs: { $max: '$millis' },
15    totalMs: { $sum: '$millis' } // total time wasted
16  }},
17
18  // Sort by total impact
19  { $sort: { totalMs: -1 } },
20  { $limit: 10 }
21 ])
22
23 slowQueries.forEach(q => {
24   console.log({
25     collection: q._id.ns,
26     queryShape: q._id.query,
27     count: q.count,
28     avgMs: Math.round(q.avgMs),
29     maxMs: q.maxMs,
30     totalImpactMs: Math.round(q.totalMs) // focus on this!
  })
})

```

```
31   })
32 })
33
34 // For each slow query: run explain to find why
35 // const result = await db.collection('users').find(slowQuery).explain('executionStats')
36 // Look for COLLSCAN, high docsExamined/nReturned ratio
37
38 // MongoDB Atlas: Performance Advisor does this automatically
39 // Shows suggested indexes based on slow query analysis
```

■ Interview Tip:

Sort slow queries by total impact (count × avg duration), not just by slowest single query. A 500ms query that runs once is less important than a 20ms query that runs 100,000 times. Focus optimization effort where total time is highest.

Index usage stats in MongoDB?

■ Answer

Index usage statistics show which indexes are being used and which are wasted.

\$indexStats aggregation:

- Shows operations count per index
- Identifies unused indexes
- Shows which indexes are being hit most

Unused indexes:

- Waste memory (part of working set)
- Slow down writes (must update on every write)
- Drop indexes with 0 usage!

Missing indexes:

- Identified by COLLSCAN in explain()
- High docsExamined vs nReturned ratio
- Atlas Performance Advisor suggests these

```

1 // View index usage statistics
2 const indexStats = await User.aggregate([ { $indexStats: {} } ])
3
4 indexStats.forEach(stat => {
5   console.log({
6     indexName: stat.name,
7     keyPattern: stat.key,
8     usageCount: stat.accesses.ops,      // queries using this index
9     sinceDate: stat.accesses.since,    // when stats were reset
10  })
11 })
12
13 // Find unused indexes
14 const unusedIndexes = indexStats.filter(s => s.accesses.ops === 0 && s.name !== '_id_')
15 console.log('Unused indexes:', unusedIndexes.map(s => s.name))
16 // Drop unused indexes to free memory and speed up writes
17 // await User.collection.dropIndex('unusedIndexName')
18
19 // Find most used indexes
20 const topIndexes = indexStats.sort((a, b) => b.accesses.ops - a.accesses.ops).slice(0, 5)
21 console.log('Top indexes:', topIndexes.map(s => ({ name: s.name, ops: s.accesses.ops })))
22
23 // Find queries NOT using any index (COLLSCAN)
24 const collscans = await db.collection('system.profile').find({
25   'execStats.stage': 'COLLSCAN',
26   'millis': { $gt: 10 } // slow COLLSCANS
27 }).sort({ millis: -1 }).limit(10).toArray()
28
29 // After adding indexes, reset stats to start fresh

```

```

30 await db.command({ resetIndexStats: 'users' }) // reset for specific collection

```

■ Interview Tip:

Run index usage analysis monthly. Drop indexes with 0 usage — they consume RAM and slow every write. In production, you can safely drop unused indexes with little risk (MongoDB will use COLLSCAN for queries that relied on them — you'll see them in slow query logs).

Data consistency in MongoDB?

■ Answer

Data consistency ensures all reads return accurate, up-to-date data.

Consistency types:

- Strong consistency — reads always see latest write (primary only)
- Eventual consistency — reads may see stale data (secondaries)
- Causal consistency — you see your own writes + their consequences

Achieving strong consistency:

- Read from primary (default in Mongoose)
- Write concern: w: 'majority', j: true
- Read concern: 'majority'

Achieving causal consistency:

- Use sessions with causal consistency enabled
- Guarantees read-your-own-writes

```

1 // Strong consistency (read from primary)
2 const account = await Account.findById(id) // always primary ■
3
4 // Eventual consistency (may read stale data from secondary)
5 const feed = await Post.find({ userId: followedUser })
6   .read('secondary') // may be milliseconds stale – acceptable for feeds
7
8 // Causal consistency with sessions
9 const session = await mongoose.startSession({
10   causalConsistency: true // read-your-writes guaranteed
11 })
12
13 // Update profile
14 await User.updateOne({ _id: userId }, { $set: { name: 'New Name' } }, { session })
15
16 // Read your own update – guaranteed to see 'New Name'
17 const user = await User.findById(userId, null, { session })
18 console.log(user.name) // 'New Name' – guaranteed even on secondary!
19
20 session.endSession()
21
22 // Write concern for financial operations
23 const FINANCIAL_WRITE_OPTS = {
24   writeConcern: { w: 'majority', j: true }
25 }
26 await Payment.create([paymentData], FINANCIAL_WRITE_OPTS)
27 // Confirmed on disk on majority of nodes – won't be rolled back
28
29 // Check if data is consistent across replica set
30 const primaryData = await Account.findById(id).read('primary')

```

```

31 const secondaryData = await Account.findById(id).read('secondary')

```

```
32 if (JSON.stringify(primaryData) !== JSON.stringify(secondaryData)) {  
33   console.warn('Replication lag detected – secondary has stale data')  
34 }
```

■ Interview Tip:

For financial applications, consistency is non-negotiable. Use w: 'majority', j: true for all financial writes, and always read from primary for balance checks. The latency cost (5-20ms extra) is worth ensuring data accuracy.

Backup strategy in MongoDB?

■ Answer

A solid backup strategy protects against data loss from hardware failures, accidental deletions, and bugs.

Backup methods:

- mongodump — logical backup (BSON export)
- Filesystem snapshots — block-level (fastest)
- MongoDB Atlas backups — managed, point-in-time restore
- Ops Manager — on-prem managed backups

Backup best practices:

- 3-2-1 rule: 3 copies, 2 media types, 1 offsite
- Test restores regularly! (monthly drills)
- Encrypt backups
- Monitor backup completion

RTO/RPO targets:

- RPO: max data loss (backup frequency)
- RTO: max restore time

```

1  // mongodump backup
2  // mongodump --uri='mongodb://user:pass@host/db' --gzip --out=/backup/$(date +%Y%m%d)
3
4  // Automated backup script
5  const { exec } = require('child_process')
6  const cron = require('node-cron')
7
8  cron.schedule('0 2 * * *', async () => { // 2 AM daily
9    const date = new Date().toISOString().slice(0, 10)
10   const backupPath = `/tmp/mongo_backup_${date}`
11
12   await new Promise((resolve, reject) => {
13     exec(
14       `mongodump --uri=${MONGO_URI} --gzip --out=${backupPath}`,
15       (error, stdout, stderr) => {
16         if (error) reject(error)
17         else resolve(stdout)
18       }
19     )
20   })
21
22   // Upload to S3 (offsite!)
23   await exec(`aws s3 sync ${backupPath} s3://myapp-backups/mongodb/${date}`)
24
25   // Clean up local temp
26   await exec(`rm -rf ${backupPath}`)
27
28   // Delete backups older than 30 days

```

```

29   await exec(`aws s3 ls s3://myapp-backups/mongodb/ | awk '{print $2}' | ...`)

```

```
30
31   console.log(`Backup completed: ${date}`)
32 })
33
34 // MongoDB Atlas: automated backups built-in
35 // - Continuous backups (every 6 hours or more frequent)
36 // - Point-in-time restore (to any second in last N days)
37 // - Cross-region backup replication
38 // Just set retention policy in Atlas UI!
```

■ Interview Tip:

Test your restore process quarterly! Most companies discover their backup is corrupt or takes 10 hours to restore only when they actually need it. Schedule a monthly restore drill: restore backup to staging, verify data integrity, measure restore time.

Restore strategy in MongoDB?

■ Answer

Restore strategy defines how you recover data after a failure or data loss incident.

Restore methods:

- mongorestore — restore from mongodump backup
- Point-in-time restore — restore to specific timestamp
- Oplog replay — apply operations to catch up

Restore scenarios:

- Accidental collection drop — restore from last backup
- Accidental data corruption — point-in-time before corruption
- Hardware failure — restore to new server

Restore time factors:

- Backup size
- Network speed (S3 → server)
- Index rebuild time
- Oplog replay time

```
1 // Restore from mongodump
2 // mongorestore --uri='mongodb://new-host/mydb' --gzip /backup/2024-01-15/
3
4 // Restore specific collection only
5 // mongorestore --uri=... --gzip --nsInclude='mydb.users' /backup/2024-01-15/
6
7 // Point-in-time restore (with oplog)
8 // 1. Restore base backup
9 // mongorestore --uri=... --gzip /backup/2024-01-15/ --oplogReplay
10 // 2. Apply oplog to specific time
11 // mongorestore --uri=... --gzip /backup/2024-01-15/ --oplogReplay
12 // --oplogLimit 1705123456 // Unix timestamp to stop at
13
14 // Automated restore verification
15 async function verifyRestore() {
16   const checks = [
17     User.countDocuments(),
18     Order.countDocuments(),
19     Product.countDocuments()
20   ]
21   const [users, orders, products] = await Promise.all(checks)
22
23   // Compare with expected counts from backup metadata
24   const expected = await BackupMetadata.findOne({ date: backupDate })
25   if (Math.abs(users - expected.userCount) > expected.userCount * 0.01) {
26     throw new Error(`User count mismatch: got ${users}, expected ~${expected.userCount}`)
27   }
28   console.log('Restore verification passed!')
```

```
29 }
```



```
30
31 // MongoDB Atlas: one-click point-in-time restore
32 // Atlas UI: Backup → select cluster → Restore → pick date/time → restore to cluster
```

■ Interview Tip:

Practice restore drills quarterly. When you actually need a restore is the worst time to discover issues. Know your RTO (how long restore takes) and RPO (how much data you lose). Atlas gives RTO < 30 min and RPO < 1 hour with continuous backups.

Disaster recovery in MongoDB?

■ Answer

Disaster recovery (DR) ensures you can restore service after catastrophic failures.

DR tiers:

- RTO 0 / RPO 0: Active-active multi-region (most expensive)
- RTO < 1h / RPO < 1h: Warm standby in another region
- RTO < 4h / RPO < 24h: Cold restore from backups (cheapest)

Multi-region replica set:

- Nodes in different regions/datacenters
- Auto-failover if entire region fails
- Slightly higher write latency (multi-region replication)

Backup-based DR:

- Regular backups to different region (S3)
- Restore to new cluster in DR region on failure

```

1 // Multi-region replica set for DR
2 // Primary: us-east-1
3 // Secondary 1: us-east-1 (same region, different AZ)
4 // Secondary 2: eu-west-1 (different region)
5
6 // rs.initiate({
7 //   _id: 'rs0',
8 //   members: [
9 //     { _id: 0, host: 'mongo1.us-east-1:27017', priority: 3 }, // preferred primary
10 //     { _id: 1, host: 'mongo2.us-east-1:27017', priority: 2 }, // US secondary
11 //     { _id: 2, host: 'mongo3.eu-west-1:27017', priority: 0 } // EU DR (no voting)
12 //   ]
13 // })
14 // priority: 0 on EU = won't become primary automatically (avoid cross-region primary!)
15
16 // DR runbook
17 async function failoverToEU() {
18   console.log('DISASTER RECOVERY: Initiating failover to EU')
19
20   // 1. Update connection string to EU node
21   process.env.MONGO_URI = process.env.MONGO_URI_EU
22
23   // 2. Force EU secondary to become primary
24   // rs.reconfigForce({ ... EU member with priority: 1 ... })
25
26   // 3. Update DNS or load balancer to point to EU
27
28   // 4. Notify team
29   await notifySlack('■■ FAILOVER COMPLETE: Now running on EU MongoDB cluster')
30 }

```

31



```
32 // MongoDB Atlas: multi-cloud/multi-region clusters
33 // Configure nodes across AWS/GCP/Azure and multiple regions
34 // Atlas handles failover automatically
35
36 // Test DR quarterly (GameDay exercises)
37 // Simulate: kill primary region, measure recovery time
```

■ **Interview Tip:**

Configure a DR secondary in a different region with priority: 0 so it won't accidentally become primary during normal operations. It stays as a hot standby. During a real disaster, manually promote it with `rs.reconfigForce()`. Atlas global clusters make this much simpler.

Security best practices in MongoDB?

■ Answer

MongoDB security requires layering multiple protections to prevent unauthorized access.

Security checklist:

- Enable authentication (required in production!)
- Use TLS for all connections
- Principle of least privilege (role-based access)
- Network access controls (IP whitelist / VPC)
- Enable auditing for sensitive operations
- Regular security patches

Common security mistakes:

- MongoDB exposed to internet without auth (Shodan!)
- Application using admin user
- No encryption in transit
- Debug mode in production

```

1 // Connection with TLS and auth
2 mongoose.connect(process.env.MONGO_URI, {
3   tls: true,
4   tlsCertificateKeyFile: '/certs/client.pem',
5   authSource: 'admin',
6   // Never expose credentials in code – use environment variables!
7 })
8
9 // Create least-privilege application user
10 // db.createUser({
11 //   user: 'app_user',
12 //   pwd: 'strong-random-password',
13 //   roles: [
14 //     { role: 'readWrite', db: 'myapp' }, // only readWrite on app db
15 //     // NOT: 'root' or 'dbAdmin'!
16 //   ]
17 // })
18
19 // Mongoose input sanitization (prevent NoSQL injection)
20 const mongoSanitize = require('express-mongo-sanitize')
21 app.use(mongoSanitize()) // strips $ and . from user input
22 // Prevents: { 'email': { '$gt': '' } } injection attacks
23
24 // Validate all inputs before DB queries
25 const loginSchema = z.object({
26   email: z.string().email(),
27   password: z.string().min(8)
28 })
29 const { email, password } = loginSchema.parse(req.body)
30 // Zod throws if input doesn't match – safe to use in query

```



```
32 // Field-level encryption (MongoDB Client-Side Field Level Encryption)
33 // Encrypt sensitive fields BEFORE sending to MongoDB
34 // MongoDB server never sees plaintext SSN/credit cards
```

■ **Interview Tip:**

Never run MongoDB without authentication in production. Thousands of exposed MongoDB databases have been wiped and held for ransom via Shodan searches. Enable auth, restrict network access to VPC/private subnet, use TLS, and use least-privilege users.

Avoid large documents in MongoDB?

■ Answer

Large documents hurt performance — they use more RAM, take longer to transfer, and hit the 16MB limit.

What causes large documents:

- Unbounded arrays (\$push without limit)
- Embedded large text or binary data
- Storing everything in one document

Strategies to avoid large documents:

- Subset pattern — embed only recent/summary data
- Reference large arrays instead of embedding
- Store binary files in GridFS or S3
- Use separate collection for heavy sub-documents

Warning signs:

- Document consistently > 100KB
- Arrays with thousands of elements
- Frequent update-induced document moves

```

1  // PROBLEM: unbounded array growth
2  // ■ Every like adds to the array – grows forever!
3  await Post.findByIdAndUpdate(postId, {
4    $push: { likedBy: userId } // array with potentially millions of userIds
5  })
6
7  // SOLUTION 1: Separate likes collection
8  await Like.create({ postId, userId, createdAt: new Date() })
9  await Post.findByIdAndUpdate(postId, { $inc: { likeCount: 1 } }) // just a counter!
10
11 // SOLUTION 2: Subset pattern – embed only recent comments
12 const postSchema = {
13   title: String,
14   commentCount: Number, // count is fast
15   recentComments: [{ // only last 5 comments embedded
16     authorId: ObjectId,
17     text: String, createdAt: Date
18   }]
19 }
20 // On new comment: push to recentComments with $slice to keep last 5
21 await Post.findByIdAndUpdate(postId, {
22   $push: { recentComments: { $each: [newComment], $slice: -5 } },
23   $inc: { commentCount: 1 }
24 })
25
26 // Check document size
27 const doc = await Post.findById(postId).lean()
28 const sizeBytes = JSON.stringify(doc).length
29 if (sizeBytes > 100 * 1024) console.warn(`Large document: ${postId} (${sizeBytes} bytes)`)
```

30  // Documents consistently > 100KB need redesign

■ Interview Tip:

The subset pattern is the solution to unbounded arrays: embed only the last N items inline and reference the full history in a separate collection. Users see the 5 most recent comments instantly (from the post doc) and can load more on demand (separate query).

Encryption in MongoDB?

■ Answer

MongoDB supports encryption at multiple levels to protect sensitive data.

Encryption layers:

- Encryption at rest — WiredTiger encryption (data files on disk)
- Encryption in transit — TLS (data over network)
- Client-side field level encryption (CSFLE) — data encrypted in app

Encryption at rest:

- WiredTiger encrypts all data files on disk
- MongoDB Enterprise + Atlas include this
- Or: encrypt at OS/filesystem level

Client-side field level encryption:

- Encrypt before sending to MongoDB
- MongoDB server never sees plaintext
- Query on encrypted fields with FHIE

```

1 // Encryption in transit (TLS)
2 mongoose.connect(process.env.MONGO_URI, {
3   tls: true,
4   tlsCAFile: '/certs/ca.pem', // CA certificate
5 })
6
7 // Client-side field level encryption (CSFLE)
8 // MongoDB Enterprise or Atlas required
9 const { MongoClient } = require('mongodb')
10 const { ClientEncryption } = require('mongodb-client-encryption')
11
12 // Create encryption key (stored in KMS like AWS KMS)
13 const client = new MongoClient(uri, {
14   autoEncryption: {
15     keyVaultNamespace: 'encryption.__keyVault',
16     kmsProviders: {
17       aws: {
18         accessKeyId: process.env.AWS_ACCESS_KEY_ID,
19         secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY
20       }
21     },
22     schemaMap: {
23       'mydb.users': {
24         bsonType: 'object',
25         properties: {
26           ssn: {
27             encrypt: {
28               bsonType: 'string',
29               algorithm: 'AEAD_AES_256_CBC_HMAC_SHA_512-Deterministic'
30             }
31           }
32         }
33       }
34     }
35   }
36 })
  
```

```
31     }
32   }
33 }
34 }
35 }
36 })
37
38 // Now SSN is automatically encrypted before storage
39 await client.db('mydb').collection('users').insertOne({ name: 'Kaushal', ssn: '123-45-6789' })
40 // MongoDB only stores encrypted ciphertext!
```

■ Interview Tip:

Client-side field level encryption is the gold standard for PCI DSS, HIPAA, and GDPR compliance — even your DBA cannot see the encrypted fields because the encryption happens in the application before data reaches the database server.

Role-based access in MongoDB?

■ Answer

MongoDB's role-based access control (RBAC) controls what operations each user can perform.

Built-in roles:

- read — can read all collections in a database
- readWrite — read + write in a database
- dbAdmin — database administration
- userAdmin — manage users and roles
- dbOwner — read + write + admin
- root — superuser (avoid in app!)

Custom roles:

- Create roles with specific collection and action permissions
- Principle of least privilege — only what's needed

Best practices:

- Create dedicated user per application
- Never use root/admin user for app

```
1 // Create application user with least privilege
2 // db.createUser({
3 //   user: 'app_readwrite',
4 //   pwd: 'securePassword123!',
5 //   roles: [{ role: 'readWrite', db: 'myapp' }] // only myapp db
6 // })
7
8 // Create read-only analytics user
9 // db.createUser({
10 //   user: 'analytics',
11 //   pwd: 'securePassword456!',
12 //   roles: [{ role: 'read', db: 'myapp' }] // read only!
13 // })
14
15 // Custom role: only specific collections
16 // db.createRole({
17 //   role: 'orderProcessor',
18 //   privileges: [
19 //     {
20 //       resource: { db: 'myapp', collection: 'orders' },
21 //       actions: ['find', 'update'] // only read and update orders
22 //     },
23 //     {
24 //       resource: { db: 'myapp', collection: 'inventory' },
25 //       actions: ['find', 'update'] // only read and update inventory
26 //     }
27 //   ],
28 //   roles: [] // no inherited roles
29 // })
```



```
30
31 // Mongoose connection with specific user
32 mongoose.connect(
33   `mongodb://app_readwrite:${process.env.DB_PASSWORD}@host/myapp?authSource=myapp`
34 )
35
36 // Audit who has what roles
37 // db.getUsers() // list all users
38 // db.getRole('orderProcessor', { showPrivileges: true }) // show role details
```

■ Interview Tip:

Create a separate MongoDB user for each application or service. Your order service should only have access to order and inventory collections — not user passwords or payment data. If the order service is compromised, the attacker can't access sensitive data.

Auditing in MongoDB?

■ Answer

MongoDB auditing logs all database operations for security compliance and forensic analysis.

What to audit:

- Authentication attempts (success and failure)
- Administrative operations (user creation, role changes)
- Data access on sensitive collections (SSN, payment data)
- Failed queries (potential attacks)

MongoDB audit log:

- Available in MongoDB Enterprise and Atlas
- Logs to file or syslog
- Filter: which operations, which users, which collections

Application-level auditing:

- Log in your application
- Mongoose middleware to track who changed what

```

1 // Application-level audit log (works in all MongoDB editions)
2 const auditLogSchema = new mongoose.Schema({
3   userId: ObjectId,
4   action: String,      // 'create', 'update', 'delete', 'view'
5   collection: String,  // 'users', 'orders', 'payments'
6   documentId: ObjectId,
7   changes: Object,     // what changed
8   ipAddress: String,
9   userAgent: String,
10  timestamp: { type: Date, default: Date.now }
11 }, { capped: { size: 100 * 1024 * 1024, max: 100000 } }) // keep last 100K audit events
12
13 // Audit sensitive operations
14 async function auditedDeleteUser(userId, requesterId, req) {
15   const user = await User.findById(userId).lean()
16   await User.findByIdAndDelete(userId)
17
18   // Always log who deleted what
19   await AuditLog.create({
20     userId: requesterId,
21     action: 'delete',
22     collection: 'users',
23     documentId: userId,
24     changes: { deleted: user }, // snapshot of deleted document
25     ipAddress: req.ip,
26     userAgent: req.headers['user-agent']
27   })
28 }
29

```

```

30 // Mongoose post-hooks for automatic auditing

```

```
31 userSchema.post('findOneAndUpdate', function(doc) {
32   if (doc) {
33     AuditLog.create({
34       action: 'update',
35       collection: 'users',
36       documentId: doc._id,
37       changes: this.getUpdate()
38     }).catch(err => logger.error('Audit log failed', err))
39   }
40 })
```

■ Interview Tip:

GDPR requires audit trails for personal data access and modifications. Build application-level auditing regardless of MongoDB edition. Log userId, action, document ID, and timestamp for every sensitive operation. This audit trail is invaluable for security incident response.

Multi-region setup in MongoDB?

■ Answer

Multi-region MongoDB setup reduces latency for global users and provides geographic redundancy.

Strategies:

- Global replica set — nodes in multiple regions
- Zone sharding — data pinned to specific regions
- Multiple clusters — independent clusters per region

Global replica set:

- Single replica set spanning multiple regions
- Pros: simple, automatic replication
- Cons: cross-region write latency (majority write concern)

GDPR consideration:

- EU user data must stay in EU
- Use zone sharding to pin EU data to EU nodes

```

1 // Global replica set configuration
2 // rs.initiate({
3 //   _id: 'globalRs',
4 //   members: [
5 //     { _id: 0, host: 'mongo1.us-east-1:27017', priority: 3 }, // US primary
6 //     { _id: 1, host: 'mongo2.us-east-1:27017', priority: 2 }, // US secondary
7 //     { _id: 2, host: 'mongo3.eu-west-1:27017', priority: 1 }, // EU secondary
8 //     { _id: 3, host: 'mongo4.ap-south-1:27017', priority: 0 }, // Asia (voting only)
9 //   ]
10 // })
11
12 // Read from nearest region
13 mongoose.connect(uri, { readPreference: 'nearest' })
14 // Mumbai users read from ap-south-1 secondary (fast!)
15 // London users read from eu-west-1 secondary (fast!)
16
17 // MongoDB Atlas Global Clusters
18 // - Geographically distributed shards
19 // - Automatic zone-based data distribution
20 // - Data sovereignty compliance built-in
21
22 // Zone sharding for GDPR
23 // sh.addShardTag('shard-eu', 'EU')
24 // sh.addTagRange('mydb.users', { region: 'EU' }, { region: 'EU~' }, 'EU')
25 // EU users always stored in EU shard
26
27 // Application: detect user region and route
28 async function createUser(userData, region) {
29   const cluster = region === 'EU' ? euCluster : globalCluster
30   return cluster.db('myapp').collection('users').insertOne({
31     ...userData,
```

```
32     region,  
33     dataResidency: region // for compliance tracking  
34   })  
35 }
```

■ **Interview Tip:**

For GDPR compliance with multi-region, use MongoDB Atlas Global Clusters — they automatically pin EU user data to EU data centers while allowing global application deployment. This is the easiest GDPR-compliant multi-region architecture.

Cloud vs self-hosted MongoDB?

■ Answer

Choosing between MongoDB Atlas (cloud managed) and self-hosted MongoDB is a strategic decision.

MongoDB Atlas (cloud managed):

- Pros: zero ops overhead, automated backups, auto-scaling, built-in monitoring
- Cons: higher cost at scale, less customization, data in cloud
- Best for: startups, teams without DevOps, fast time-to-market

Self-hosted (EC2/bare metal):

- Pros: more control, cheaper at scale, custom tuning
- Cons: ops burden, manual backups, manual scaling
- Best for: large teams with DevOps, strict data control, cost optimization

Hybrid:

- Development/staging on Atlas
- Production self-hosted for cost

```

1 // Atlas connection (cloud managed)
2 // Atlas handles: backups, monitoring, scaling, HA, security patches
3 mongoose.connect(
4   'mongodb+srv://user:pass@cluster.mongodb.net/mydb?retryWrites=true&w=majority'
5 )
6 // Free tier: M0 (512MB), Shared: M2/M5, Dedicated: M10+
7
8 // Atlas features:
9 // - Performance Advisor: suggests indexes automatically
10 // - Atlas Search: full-text search built-in
11 // - Data API: REST API without a server
12 // - Atlas Functions: serverless JavaScript
13 // - Charts: data visualization
14 // - Online Archive: auto-archive to S3
15
16 // Self-hosted connection
17 mongoose.connect(
18   'mongodb://user:pass@mongo1:27017,mongo2:27017,mongo3:27017/mydb?replicaSet=rs0'
19 )
20 // You handle: OS updates, MongoDB updates, monitoring, backups, scaling
21
22 // Cost comparison (rough estimate):
23 // Atlas M10 (2 vCPU, 2GB RAM): ~$60/month
24 // EC2 t3.small (2 vCPU, 2GB RAM): ~$15/month (x3 for HA = $45)
25 // But EC2: add engineering time, monitoring setup, backup scripts...
26 // True cost of self-hosted >> infra cost!
27
28 // Recommendation: Atlas for < 500GB and < 50M documents
29 // Self-hosted when Atlas costs > $5K/month and you have DevOps team

```

■ Interview Tip:

The true cost of self-hosted includes engineering time for ops: backup monitoring, security patches, replica set management, capacity planning. For most startups and mid-size companies, Atlas is cheaper in total cost of ownership even if the bill is higher.

Real-world design patterns in MongoDB?

■ Answer

Battle-tested patterns for common MongoDB design challenges.

Core patterns:

- Computed — pre-store aggregated values (counts, sums)
- Subset — embed only the most-used subset of data
- Bucket — group time-series into buckets for efficiency
- Outlier — handle viral/celebrity content separately
- Extended reference — embed frequently-used fields from referenced doc
- Schema versioning — track schema version in each document

Anti-patterns to avoid:

- Massive arrays (> 1000 elements embedded)
- Bloated documents (> 100KB)
- Unnecessary indexes
- Using MongoDB as an application cache (use Redis)

```

1 // COMPUTED PATTERN: pre-store aggregated values
2 const postSchema = {
3   title: String,
4   likeCount: { type: Number, default: 0 }, // increment atomically
5   commentCount: { type: Number, default: 0 }, // increment atomically
6   viewCount: { type: Number, default: 0 } // increment on each view
7 }
8 await Post.findByIdAndUpdate(postId, { $inc: { likeCount: 1 } })
9 // Fast read: post.likeCount – no aggregation query needed
10
11 // SUBSET PATTERN: embed only recent/relevant data
12 const productSchema = {
13   name: String, price: Number,
14   // Embed top 3 reviews – covers 90% of use case
15   topReviews: [{ rating: Number, text: String, user: String }], // max 3
16   reviewCount: Number, avgRating: Number // computed
17 }
18
19 // BUCKET PATTERN: time-series efficiency
20 const metricBucketSchema = {
21   sensorId: String,
22   date: Date, // hour bucket
23   count: Number,
24   measurements: [{ ts: Date, value: Number }] // up to 60 readings/bucket
25 }
26 // 1 document per sensor per hour vs 60 documents per sensor per hour
27 // 60x fewer documents, 60x fewer index entries!
28
29 // OUTLIER PATTERN: handle viral posts
30 const viralPost = {

```

```

31   title: String,

```

```
32   likeCount: 5000000,  
33   hasExtendedLikes: true // flag for outlier  
34 }  
35 // Regular post: embed all 50 likedBy IDs inline  
36 // Viral post: separate likes collection (50M IDs won't fit inline)
```

■ Interview Tip:

These patterns are the difference between a MongoDB schema that works at prototype scale and one that works at production scale. The Computed pattern (pre-store counts) is especially important — compute on write (once), read fast (millions of times).

MongoDB 64 Q — Complete! ■

■ All 64 questions with detailed answers + code!

Follow for Next: Final full-stack 950+ (Q & A)

■ SAVE — your MongoDB bible

■■ REPOST — help your dev community

■ COMMENT 'JS950' — get all 950 Q PDF

■ FOLLOW Kaushal Singh — daily prep

Kaushal Singh

Full-Stack Dev | MERN + React Native | Next.js

1M+ Impressions | React · React Native · Express · Next · Node · MongoDB · Interview Prep daily

Section 7/10 done | Next: Final full-stack 950+ Q | Comment 'JS950'